Database — leerboek V2

LEERBOEK V2 · DATABASES & MODELLERING · DEEL 1: MODELLEREN · 15 HOOFDSTUKKEN · 14 UITGEWERKTE VOORBEELDEN · 26 OKTOBER 2026

Bron: de lessen 1, 2, 3 en 12 van Databases & modellering. Het voorbeeld in dit boek is Smaakvol, een receptensite. Het staat los van de oefeningen van de cursus en van je project.

> **OPMERKING**
>
> **Nieuw in versie 2.** Elk hoofdstuk van 1 tot 14 heeft een uitgewerkt voorbeeld, vlak voor Voorspel de uitkomst: van een korte situatie, stap voor stap, naar het resultaat. De lijst met links staat bij Hoe je dit gebruikt, onder Uitgewerkte voorbeelden.
>
> **Wat gelijk bleef.** De uitleg, de voorspelvragen, de leerwijzer, de keuzegids en de index. Versie 1 blijft bestaan.

Inhoudsopgave

Hoe je dit gebruikt 2
Inhoud 5
Leerwijzer per les 6
Keuzegids 7

Deel 1 — Modelleren 9

1. Data, tabellen en databases 10
2. Conceptueel, logisch en fysiek 16
3. Entiteiten en attributen 24
4. Naamgeving 30
5. Sleutels 36
6. Relaties en foreign keys 41
7. Aard, connectiviteit en relatiebeschrijving 48
8. Soorten entiteiten 54
9. Datatypes en zelfgemaakte types 60
10. NA en NNA 68
11. DTC, DTR en DTN 74
12. Recursieve relaties 80
13. Tijdsafhankelijke attributen 87
14. Overerving 93
15. Je model, les per les 100

Veelgemaakte verwarringen 109

Index 112

Hoe je dit gebruikt

Dit leerboek zet de leerstof van Databases op een rij, zodat je ze snel terugvindt. Wat de dia's over verschillende lessen verspreiden, staat hier per onderwerp bij elkaar. Waar de dia's een begrip enkel tonen, staat hier ook waarom het zo is, en waar je op let.

Elk hoofdstuk is volledig op zich. Wordt iets pas in een ander hoofdstuk volledig uitgelegd, dan staat er een verwijzing.

Deze versie bevat deel 1: **modelleren**. Dat is de leerstof van les 1, 2, 3 en 12. De andere delen volgen in een nieuwe versie:

- deel 2: tabellen maken in SQL Server (les 4);
- deel 3: gegevens opvragen met SQL (les 5 tot 11);
- deel 4: documentdatabases met MongoDB (les 13 en 14).

Er zijn drie soorten hoofdstukken.

- Een **begrip** legt een idee uit dat in elk model terugkomt: wat een tabel is, en waarom een model in lagen gebouwd wordt.
- Een **bouwsteen** is iets wat je in een ERD tekent of noteert: een sleutel, een relatie, een datatype, een delete-optie. Elke bouwsteen volgt dezelfde stappen: wat het is, hoe het eruitziet in een ERD, hoe je beslist, waar je op let, en de veelgemaakte fouten.
- Een **special topic** is een situatie waarvoor de cursus meerdere oplossingen geeft, elk met voor- en nadelen. Die hoofdstukken zetten de opties naast elkaar.

Het laatste hoofdstuk, **Je model, les per les**, laat één model groeien van les 1 tot les 12. Zo zie je wat er na elke les bijkomt.

Nieuw in versie 2: een uitgewerkt voorbeeld in elk hoofdstuk van 1 tot 14, vlak voor *Voorspel de uitkomst*. Het vertrekt van een korte situatie, en toont stap voor stap hoe je tot het resultaat komt. Elk voorbeeld heeft dezelfde opbouw:

- **Situatie**: wat de opdrachtgever vraagt, in een blauwgroen kader;
- **Stap 1, 2, …**: wat je beslist, en waarom;
- **Het resultaat**: meestal een ERD in de notatie van de cursus;
- **Waarom zo**: de regels uit de les die het verklaren;
- **Zo controleer je het zelf**: vragen om je eigen werk mee na te lopen.

In de tabellen staat ✓ voor wat klopt, ✗ voor wat niet klopt, en ½ voor wat maar half klopt.

Elk hoofdstuk eindigt met **Voorspel de uitkomst**. Lees de situatie, beslis wat het antwoord is, en kijk pas dan naar het antwoord, op de laatste pagina van het hoofdstuk. Uitleg lezen voelt als begrijpen, maar pas als je een uitkomst juist voorspelt, weet je of het zit.

Kom je een begrip tegen en wil je weten wat het betekent, gebruik dan de index achteraan.

Uitgewerkte voorbeelden

Alle voorbeelden gaan over Smaakvol: kookworkshops, kookboeken en recepten. Ze staan los van je project.

Hoofdstuk	Voorbeeld	Wat je ziet
1	Van een lijstje naar een tabel	rijen, kolommen, één waarde per cel, een primaire sleutel
2	Eén beschrijving, drie niveaus	hetzelfde model, conceptueel, logisch en fysiek
3	Van beschrijving naar entiteit	een entiteit, haar attributen, een samengesteld gegeven, een procesgegeven
4	Een ontwerp opkuisen	elke naam getoetst aan de naamregels
5	De sleutels van een kookboek	kandidaatsleutels, de PK en een AK
6	Waar komt de foreign key?	één-op-veel, veel-op-veel en een associatie-entiteit
7	Multipliciteiten en beschrijvingen	aard, connectiviteit en de relatiebeschrijving
8	De soort van elke entiteit	kern, karakteristiek en associatie, met de drie vragen
9	Datatypes en een vaste lijst	datatypes, een enumeratie en een karakteristieke entiteit
10	NA of NNA voor elk attribuut	NA en NNA, ook bij foreign keys, en `0` tegenover `NULL`
11	Een delete-optie voor elke foreign key	DTC, DTR en DTN, met wat er met je gegevens gebeurt
12	Varianten van een recept	een recursieve relatie
13	Een prijs die verandert	een tijdsafhankelijk attribuut met een eigen entiteit
14	Workshops op locatie en online	de drie opties van overerving naast elkaar

Volg je de lessen, dan zegt de leerwijzer welke hoofdstukken bij elke les horen.

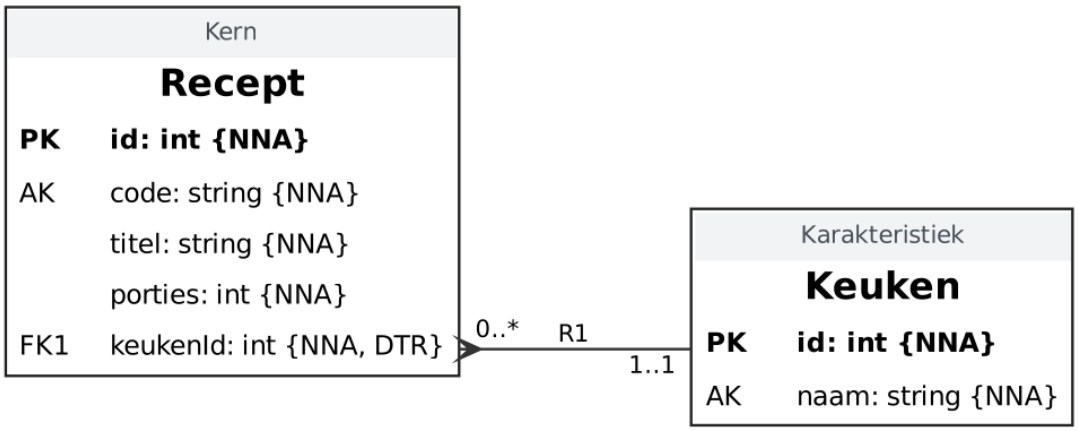
Kleuren en labels

Label	Betekenis
BEGRIP	een idee dat in elk model terugkomt
BOUWSTEEN	iets wat je tekent of noteert in een ERD
SPECIAL TOPIC	een situatie met meerdere oplossingen
LES PER LES	één model dat meegroeit met de lessen

> **OPMERKING**
>
> Het voorbeeld in dit boek is een receptensite, **Smaakvol**. Leden plaatsen er recepten, met ingrediënten, een keuken en categorieën, en geven elkaars recepten een beoordeling. Het is hetzelfde domein als in het C#-leerboek, maar het staat los van de oefeningen van de cursus en van je project.

Zo lees je een ERD in dit boek

De ERD's in dit boek volgen de notatie van de cursus, zoals je ze in draw.io tekent.



Een stukje ERD met de notatie van de cursus.

Wat je ziet	Wat het betekent	Hoofdstuk
de naam bovenaan, met een hoofdletter	de entiteit: later een tabel	3
Kern, *Associatie* of *Karakteristiek* boven de naam	de soort van de entiteit	8
`PK`	de primaire sleutel, altijd `id`	5
`AK`	een alternatieve sleutel	5
`FK1`, `FK2`, …	een foreign key, met het nummer van zijn relatie	6
`naam: type`	een attribuut met zijn datatype, na een dubbelpunt	9
`{NNA}` of `{NA}`	of het attribuut altijd ingevuld moet zijn	10
`{NNA, DTR}` bij een foreign key	NA of NNA, en de delete-optie	11
`R1`, `R2`, … bij een lijn	het nummer van de relatie	6
`0..*`, `1..1`, … aan de uiteinden	de aard en de connectiviteit van de relatie	7
een kraaienpoot aan het uiteinde	de veel-kant van de relatie	7

Inhoud

Deel	Hoofdstuk	Wat je leert
Vooraan	Uitgewerkte voorbeelden	per hoofdstuk één voorbeeld, stap voor stap
	Leerwijzer per les	welke hoofdstukken bij welke les horen
	Keuzegids	welke keuze je neemt, en waar je iets vindt
1. Modelleren	1. Data, tabellen en databases	wat een relationele database is, en de regels van een tabel
	2. Conceptueel, logisch en fysiek	de drie niveaus van een model, en de synthesemethode
	3. Entiteiten en attributen	waarover je gegevens bijhoudt, en welke
	4. Naamgeving	hoe je entiteiten, attributen en sleutels noemt
	5. Sleutels	primaire, kandidaat-, surrogaat- en alternatieve sleutels
	6. Relaties en foreign keys	één-op-één, één-op-veel, veel-op-veel, en waar de FK staat
	7. Aard, connectiviteit en relatiebeschrijving	`0..1`, `1..*` en de zinnen bij elke relatie
	8. Soorten entiteiten	kern, associatie en karakteristiek
	9. Datatypes en zelfgemaakte types	het datatype, de enumeratie en de karakteristieke entiteit
	10. NA en NNA	mag een attribuut leeg blijven of niet
	11. DTC, DTR en DTN	wat er gebeurt als je een rij verwijdert waarnaar verwezen wordt
	12. Recursieve relaties	een entiteit die naar zichzelf verwijst
	13. Tijdsafhankelijke attributen	een waarde die verandert, met of zonder geschiedenis
	14. Overerving	een algemeen type met specifieke soorten
	15. Je model, les per les	één model, van les 1 tot les 12
2. Tabellen maken	volgt in een volgende versie	`CREATE TABLE`, constraints, volgorde van aanmaken (les 4)
3. Gegevens opvragen	volgt in een volgende versie	`SELECT`, functies, joins, set-functies, subqueries, `GROUP BY` (les 5 tot 11)
4. Documentdatabases	volgt in een volgende versie	MongoDB: queries, embedden of refereren, schema's (les 13 en 14)
Achteraan	Veelgemaakte verwarringen	begrippen die op elkaar lijken
	Index	alles alfabetisch, met elk hoofdstuk waar het voorkomt

Leerwijzer per les

Welke hoofdstukken horen bij welke les? Lees ze vóór of naast de les. Hoofdstuk 15 volgt elke les mee.

Les	Onderwerp	Hoofdstukken
1	Entiteiten en attributen	1, 2, 3, 4, 15
2	Sleutels en relaties	5, 6, 7, 8, 15
3	Datatypes en specificaties	9, 10, 11, 15
4 tot 11	SQL	volgt in deel 2 en 3
12	Special topics	12, 13, 14, 15
13 en 14	Documentdatabases	volgt in deel 4

> **OPMERKING**
>
> De cursus geeft les 12 na de SQL-lessen, maar het is leerstof over modelleren. Daarom staat ze in dit boek bij deel 1.

Keuzegids

Welke keuze neem ik?

Je twijfelt tussen…	Vuistregel	Hoofdstuk
één kolom of twee, zoals `naam` of `voornaam` en `familienaam`	wil je de delen ooit apart gebruiken, sorteren of zoeken, dan splits je	3
een attribuut bewaren of berekenen, zoals een leeftijd	wat je kan berekenen uit andere gegevens, bewaar je niet	3
een natuurlijke sleutel of `id` als PK	altijd `id`, een getal zonder betekenis; de natuurlijke sleutel wordt een AK	5
waar de foreign key komt	aan de veel-kant, de kant met de `*`	6
twee entiteiten met een één-op-éénrelatie	meestal samenvoegen tot één entiteit	6
een veel-op-veelrelatie in een fysiek model	altijd een associatie-entiteit met twee één-op-veelrelaties	6
aard 0 of 1	kan het ene bestaan zonder het andere? Dan is de aard 0	7
een getal of een tekst als datatype	enkel een getal als je ermee rekent	9
een enumeratie of een karakteristieke entiteit	een enumeratie als de lijst klein en vast is; een karakteristieke entiteit als ze groeit of eigen gegevens krijgt	9
NA of NNA bij een foreign key	lees het af uit de aard: aard 1 is NNA, aard 0 is NA	10
DTC, DTR of DTN	DTR is de standaard; DTC bijna enkel bij een associatie-entiteit; DTN enkel bij aard 0	11
een hiërarchisch model of een recursieve relatie	vaste niveaus met eigen gegevens: hiërarchisch; veel gelijkaardige niveaus: recursief	12
enkel de actuele waarde of ook de geschiedenis	heb je de oude waarden nodig, dan bewaar je ze; vaak wijzigen vraagt een eigen entiteit	13
één entiteit, één per klasse, of joined tables	weinig eigen attributen per soort: één entiteit; veel en het groeit: joined tables	14

Waar vind ik…

Ik zoek…	Hoofdstuk
de regels waaraan een tabel voldoet	1
wat er in een conceptueel, logisch en fysiek model staat	2
de stappen van de synthesemethode	2, 15
de naamregels van de cursus	4
de ERD-checklists uit de dia's	4, 5, 6, 15
hoe je een relatie beschrijft in zinnen	7
hoe je de soort van een entiteit herkent	8
welke datatypes je op logisch niveau gebruikt	9
hoe een delete-optie je gegevens verandert	11
waarom SQL Server geen DTC of DTN toelaat bij een recursieve relatie	12
een volledig model, van begin tot eind	15

Deel 1 — Modelleren

Voor je één tabel maakt, beslis je waarover je gegevens bijhoudt, hoe die samenhangen en wat elk gegeven mag zijn. Dat leg je vast in een ERD. Een fout in het model werkt door in alles wat erop volgt: de tabellen, de queries en de software.

1. Data, tabellen en databases

BEGRIP

Wat het is

Data zijn losse gegevens: cijfers of tekst die je verzamelt, zonder uitleg erbij. Een jaar lang elke week de prijs van bloem noteren, levert data op.

Informatie ontstaat als je die data ordent en interpreteert. Uit dezelfde prijzen leer je bijvoorbeeld dat bloem dit jaar duurder werd.

Een **database** is een verzameling gegevens die blijven bestaan, ook als het programma stopt. Ze wordt gebruikt door de software van een organisatie, en beheerd door een **DBMS**, een databasemanagementsysteem. SQL Server, PostgreSQL en MongoDB zijn DBMS'en.

Een **relationele database** bewaart alle gegevens in tabellen. Het relationele model komt van E.F. Codd, in 1970. In een tabel:

- bevat een **rij**, of **record**, de gegevens van één ding: één recept, één lid;
- bevat een **kolom**, of **attribuut**, één soort gegeven voor elke rij: de titel van elk recept.

Tabellen verwijzen naar elkaar via een kolom die naar een rij in een andere tabel wijst. Dat is een **foreign key**. Zie hoofdstuk 6.

Hoe het eruitziet

De tabel `Recept` van Smaakvol:

id	code	titel	bereidingstijd	porties
1	R-0001	Lasagne	90	6
2	R-0002	Pad thai	30	2
3	R-0003	Tomatensoep	45	4

Elke rij is één recept. Elke kolom is één gegeven dat je van elk recept bijhoudt.

De regels van een tabel

Regel	Wat het betekent
de rijen hebben geen volgorde	een tabel is een verzameling rijen. Dezelfde rijen in een andere volgorde zijn dezelfde tabel
de kolommen hebben geen volgorde	`titel` voor of na `porties`: het blijft dezelfde tabel
twee identieke rijen mogen niet	elke rij moet van alle andere te onderscheiden zijn
elke waarde in een kolom is van dezelfde soort	in `titel` staan enkel titels, in `porties` enkel aantallen
de kruising van een rij en een kolom bevat één waarde	één recept heeft in `titel` één titel, niet twee

De laatste regel is de belangrijkste voor het modelleren. Heeft een recept meerdere ingrediënten, dan kan je die niet allemaal in één cel zetten. Daarvoor heb je een tweede tabel nodig. Zie hoofdstuk 6.

De primaire sleutel

Twee rijen mogen nooit verwisseld kunnen worden. Daarom heeft elke tabel een kolom die elke rij uniek maakt: de **primaire sleutel**, of PK. In deze cursus:

- heet de primaire sleutel altijd `id`;
- is het een geheel getal;
- is ze altijd ingevuld;
- heeft ze geen betekenis: het getal zegt niets over het recept;
- is ze minimaal: één kolom, geen combinatie.

Waarom het zo moet, staat in hoofdstuk 5.

> **OPMERKING**
>
> **Een ERD tegenover een database.** In het ERD teken je de structuur: de entiteit `Recept` met haar attributen. In de database staan de rijen: echte recepten. Het ERD zegt welke kolommen er zijn en wat erin mag. De gegevens zelf komen pas met SQL, in deel 2.

Waarom modelleren zo belangrijk is

De cursus noemt twee redenen:

- **het hefboomeffect**: een fout in het model heeft later grote gevolgen. Elke tabel, query en applicatie bouwt erop verder;
- **bondigheid**: bewaar je een waarde twee keer, dan zit je eraan vast. Verander je ze op één plaats, dan klopt de andere niet meer.

Neem dus de tijd voor je model. Een model snel opstellen zonder doordachte keuzes, kost je later veel werk aan je tabellen en je software.

De cursus zegt ook: databaseontwerp is een kunst, geen wetenschap. Er zijn richtlijnen, maar ook uitzonderingen. Je maakt een systeem voor iemand anders: stel vragen, en zet je eigen beslissingen ter discussie.

Veelgemaakte fouten

> LET OP
>
> - **Meerdere waarden in één cel**, zoals *bloem, eieren, melk* in één kolom `ingredienten`. Een cel bevat één waarde.
> - **Data en informatie door elkaar halen.** Een database bewaart data. Informatie haal je eruit met queries.
> - **Een kolom zonder vaste soort**, zoals een kolom `extra` waarin soms een datum en soms een naam staat.

Uitgewerkt voorbeeld: van een lijstje naar een tabel

> **SITUATIE**
>
> Smaakvol houdt een lijst bij van kookboeken waaruit leden recepten overnemen. Tot nu toe stond die lijst in een notitie:
>
> *Soepen voor elk seizoen, Marie Peeters, 2019, ISBN 9781000000017. Thema's: soep, seizoen.*
>
> *Snel en gezond, Tom Janssens, 2021, ISBN 9781000000024. Thema's: snel, vegetarisch, gezond.*
>
> *Soepen voor elk seizoen, Marie Peeters, 2023, herziene druk, ISBN 9781000000031. Thema's: soep, seizoen.*
>
> Maak er een tabel van.

Stap 1. Wat is één rij? Elke regel van de notitie gaat over één kookboek. Eén kookboek wordt dus één rij. De eerste en de derde regel hebben dezelfde titel, maar het zijn twee drukken, met een ander jaar en een ander ISBN. Het blijven twee rijen.

Stap 2. Welke kolommen? Lees wat je van elk boek weet: een titel, een auteur, een jaar, een ISBN en thema's. Elk gegeven dat elk boek één keer heeft, wordt een kolom. De vermelding *herziene druk* laat je hier weg: het jaar en het ISBN zeggen al dat het een andere druk is. Wil Smaakvol ze toch tonen, dan wordt het een eigen kolom.

Stap 3. Eén waarde per cel? Bij de thema's lukt dat niet: *Snel en gezond* heeft er drie. Drie thema's in één cel zijn drie waarden, en dat mag niet. De thema's worden dus geen kolom van deze tabel. Een boek heeft meerdere thema's, en een thema komt in meerdere boeken voor: dat is een veel-op-veelrelatie. De thema's krijgen een eigen tabel, met een tussentabel die boeken en thema's koppelt. Hoe dat gaat, staat in hoofdstuk 6.

Stap 4. Wat maakt elke rij uniek? Geef elke rij een `id`: een geheel getal zonder betekenis.

Het resultaat.

id	titel	auteur	jaar	isbn
1	Soepen voor elk seizoen	Marie Peeters	2019	9781000000017
2	Snel en gezond	Tom Janssens	2021	9781000000024
3	Soepen voor elk seizoen	Marie Peeters	2023	9781000000031

Waarom zo.

Regel van een tabel	Hoe het hier klopt
twee identieke rijen mogen niet	✓ rij 1 en 3 verschillen in `jaar`, `isbn` en `id`
elke waarde in een kolom is van dezelfde soort	✓ in `jaar` staan enkel jaartallen
de kruising van een rij en een kolom bevat één waarde	✓ de thema's staan er niet meer in
de rijen en kolommen hebben geen volgorde	✓ niets in de tabel hangt van de volgorde af

Wat je nog opvalt. *Marie Peeters* staat twee keer. Dat is nog geen fout: een tabel mag dezelfde waarde in meerdere rijen hebben. Of een auteur een eigen tabel verdient, beslis je bij het modelleren. Zie hoofdstuk 3, *Een gegeven op de juiste plaats*.

Zo controleer je het zelf.

- Kan je in één zin zeggen wat één rij is? Hier: één kookboek.
- Heeft elke cel precies één waarde?
- Staat in elke kolom één soort gegeven?
- Is er een kolom die elke rij uniek maakt?

Voorspel de uitkomst

Vraag 1

Een tabel `Lid` heeft twee rijen met exact dezelfde voornaam, familienaam en e-mailadres, en geen kolom `id`. Welke regel van een tabel wordt overtreden?

Toon het antwoord ›

Vraag 2

In de tabel `Recept` staat een kolom `allergenen` met de waarde *gluten, ei, melk*. Welke regel wordt overtreden?

Toon het antwoord ›

Vraag 3

Je zet de rijen van `Recept` in een andere volgorde, en de kolom `porties` voor `titel`. Is het nog dezelfde tabel?

Toon het antwoord ›

Antwoorden

VRAAG 1

Twee identieke rijen mogen niet. Zonder iets wat ze onderscheidt, kan de database de twee leden niet uit elkaar houden. Daarom heeft elke tabel een primaire sleutel.

‹ Terug naar de vraag

VRAAG 2

De kruising van een rij en een kolom bevat één waarde. Drie allergenen in één cel zijn drie waarden. Een recept kan meerdere allergenen hebben, en een allergeen komt in meerdere recepten voor: dat wordt een veel-op-veelrelatie. Zie hoofdstuk 6.

‹ Terug naar de vraag

VRAAG 3

Ja. Rijen en kolommen hebben geen volgorde. Wil je ze in een bepaalde volgorde zien, dan vraag je dat in je query.

‹ Terug naar de vraag

2. Conceptueel, logisch en fysiek

BEGRIP

Wat het is

Een **datamodel** is een tekening, geen code. Het is het resultaat van een analyse: welke gegevens er zijn, en hoe ze samenhangen. Het dient ook om te praten met wie de software bestelt, en het is de bron waaruit je de database bouwt.

De cursus tekent datamodellen als **ERD**: een *entity relationship diagram*. Er bestaan meerdere notaties, zoals Chen, Crow's Foot, UML, IDEF1X, Bachman en Martin. De cursus gebruikt een eigen mengvorm, die je in draw.io tekent. Zie *Zo lees je een ERD in dit boek*, vooraan.

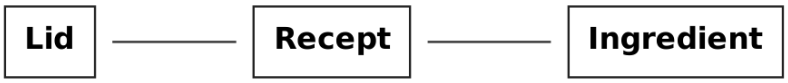
Je bouwt een model in drie stappen, van ruw naar volledig: **conceptueel**, **logisch** en **fysiek**.

Wat er op elk niveau staat

	Conceptueel	Logisch	Fysiek
de namen van de entiteiten	ja	ja	ja
relaties	ja	ja	ja
veel-op-veelrelaties toegelaten	ja	ja, of al vervangen	nee
attributen	nee	ja	ja
datatypes	nee	algemene, als je wil	die van het DBMS
primaire en foreign keys	nee	als je wil	ja
alternatieve sleutels	nee	nee	ja
NA en NNA	nee	nee	ja
relatiespecificaties, zoals DTC, DTR en DTN	nee	nee	ja

> **OPMERKING**
>
> De dia's zijn op twee plaatsen niet eenduidig. Hun overzichtstabel zet bij een conceptueel model ook attributen, *maar niet genormaliseerd*, en noemt relatiespecificaties optioneel in een logisch model. De dia's over elk niveau zeggen: geen attributen in het conceptuele model, en geen relatiespecificaties in het logische. Dit boek volgt die tweede uitleg. Twijfel je bij een opdracht, vraag dan na wat je leerkracht verwacht.

Conceptueel is het overzicht. Enkel de kernentiteiten, de dingen waar het om draait, met lijnen tussen wat samenhangt. Iedereen moet het kunnen lezen, ook wie niet programmeert.



Het conceptuele model van Smaakvol: enkel kernentiteiten en lijnen.

Logisch voegt de details toe: attributen, en eventueel sleutels en algemene datatypes. Het hangt nog niet af van een bepaald DBMS.

Fysiek is klaar om te bouwen in één bepaald DBMS: alle sleutels, de datatypes van dat DBMS, NA en NNA, de delete-opties, en namen volgens de regels van dat DBMS. Een veel-op-veelrelatie is hier altijd vervangen door een associatie-entiteit.

De synthesemethode

De cursus bouwt een model op met de **synthesemethode**: van een beschrijving in gewone taal naar een model, stap voor stap.

1. Zoek de **kernentiteiten**: waarover houd je gegevens bij? Teken het conceptuele model.
2. Geef elke entiteit haar **attributen**.
3. Leg de **relaties** vast, met hun multipliciteit en een beschrijving.
4. Vervang elke **veel-op-veelrelatie** door een associatie-entiteit.
5. Voeg de **sleutels** toe: PK, FK en AK.
6. Geef elk attribuut zijn **datatype** en NA of NNA.
7. Kies voor elke foreign key een **delete-optie**.

Het voorbeeld in de les is de computerwinkel van Harry en Saartje, die de bestellingen van klanten willen automatiseren. De kernentiteiten zijn `Klant`, `Product` en `Order`. In het logische model krijgt `Product` een attribuut `naam`, niet `productNaam`. `Order` krijgt `orderdatum`, omdat *datum* alleen niet zegt welke datum bedoeld is.

> **BELANGRIJK**
>
> **Zo weinig mogelijk relaties.** In het voorbeeld uit de les is er geen lijn tussen `Product` en `Klant`. Welke klant welk product kocht, vind je via `Order`. Bestaat er al een pad tussen twee entiteiten, dan is een tweede pad meestal overbodig, en kan het zelfs iets anders zeggen dan het eerste.

Hoofdstuk 15 doorloopt de synthesemethode volledig, voor Smaakvol.

Veelgemaakte fouten

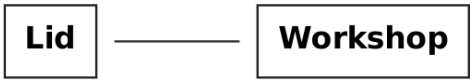
LET OP

- **In het conceptuele model al attributen tekenen.** Op dat niveau beslis je enkel welke entiteiten er zijn, en wat samenhangt.
- **Een veel-op-veelrelatie laten staan in het fysieke model.** Een database kan ze niet bewaren zonder associatie-entiteit.
- **Te vroeg aan SQL beginnen.** Een fout in het model kost je later meer werk dan de tijd die je eraan bespaarde.
- **Een extra lijn tekenen voor iets wat al via een andere weg bereikbaar is.**

Uitgewerkt voorbeeld: één beschrijving, drie niveaus

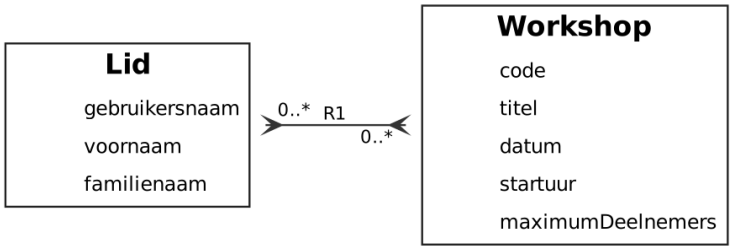
> **SITUATIE**
>
> Smaakvol organiseert kookworkshops. Een workshop heeft een code, zoals *WS-0142*, een titel, een datum, een startuur en een maximum aantal deelnemers. Van een lid kent Smaakvol een gebruikersnaam, die uniek is, en een voornaam en een familienaam. Leden schrijven zich in voor workshops. Een lid kan zich voor veel workshops inschrijven, en een workshop heeft veel deelnemers. Smaakvol wil weten wanneer elk lid zich inschreef.
>
> Maak het conceptuele, het logische en het fysieke model, voor SQL Server.

Stap 1. Conceptueel: waarover houd je gegevens bij? Er zijn twee kernentiteiten: `Lid` en `Workshop`. Ze hangen samen, want leden schrijven zich in voor workshops. Op dit niveau teken je enkel de namen en een lijn. Geen attributen, geen sleutels.



Het conceptuele model: twee kernentiteiten en een lijn.

Stap 2. Logisch: de attributen en de relatie. Elke entiteit krijgt haar attributen. Bij de lijn zet je de multipliciteit. Een lid kan zich voor geen, één of meerdere workshops inschrijven, en een workshop kan geen, één of meerdere leden hebben. Dat is een veel-op-veelrelatie, en op dit niveau mag ze nog blijven staan.

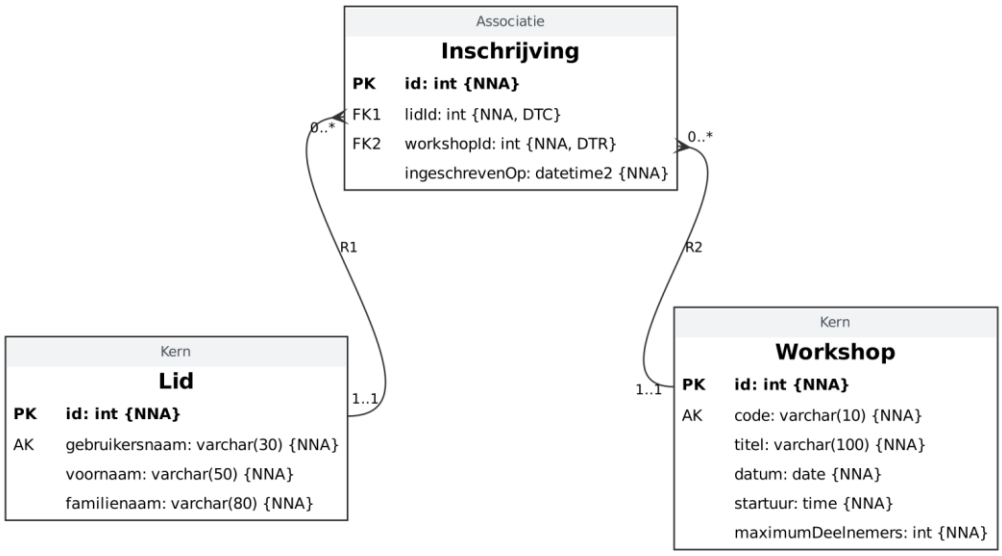


Het logische model: attributen, en een veel-op-veelrelatie.

Waar komt *wanneer het lid zich inschreef*? Niet in `Lid`: een lid heeft veel inschrijvingen, elk met een eigen moment. Niet in `Workshop`: een workshop heeft veel deelnemers. Het hoort bij de combinatie van één lid en één workshop. Die combinatie bestaat pas in de volgende stap.

Stap 3. Fysiek: klaar voor SQL Server. Nu mag er geen veel-op-veelrelatie meer staan.

- De relatie wordt een **associatie-entiteit** `Inschrijving`, met een foreign key naar elk van beide. Het inschrijfmoment krijgt daar zijn plaats: `ingeschrevenOp`.
- Elke entiteit krijgt een **primaire sleutel** `id`. `gebruikersnaam` en `code` zijn uniek, en worden **alternatieve sleutels**.
- Elk attribuut krijgt een **datatype van SQL Server**, en **NA of NNA**.
- Elke foreign key krijgt een **delete-optie**. Wordt een lid verwijderd, dan hebben zijn inschrijvingen geen betekenis meer: DTC. Een workshop met inschrijvingen verwijder je niet zomaar, want de leden moeten eerst verwittigd worden: DTR.



Het fysieke model: geen veel-op-veel meer, met alle sleutels, datatypes en specificaties.

> ***R1**: Een inschrijving hoort bij precies één lid. Een lid heeft geen, één of meerdere inschrijvingen.*
>
> ***R2**: Een inschrijving hoort bij precies één workshop. Een workshop heeft geen, één of meerdere inschrijvingen.*

Waarom zo.

	Conceptueel	Logisch	Fysiek
entiteiten	`Lid`, `Workshop`	`Lid`, `Workshop`	+ `Inschrijving`
attributen	✗ nog niet	✓	✓
veel-op-veel	✓ mag	✓ mag	✗ vervangen
sleutels	✗	✗ hier nog niet	✓ PK, FK, AK
datatypes	✗	✗ hier nog niet	✓ SQL Server
NA, NNA, delete-opties	✗	✗	✓

Dit voorbeeld houdt het logische model kaal. Een logisch model mag ook al sleutels en algemene datatypes tonen. In de volgende hoofdstukken zie je ERD's met algemene datatypes zoals `string`, samen met NA, NNA en delete-opties. Zo doen ook de voorbeeldmodellen van de cursus: het is een fysiek model waarin de datatypes nog algemeen genoteerd zijn. Zie hoofdstuk 9.

Zo controleer je het zelf.

- Staat er in je conceptuele model iets anders dan namen en lijnen? Dan zit je al op het logische niveau.
- Staat er in je fysieke model nog een lijn met een `*` aan beide kanten? Dan is het nog niet fysiek.
- Heeft elk gegeven uit de beschrijving een plaats? Hier: ook het inschrijfmoment.

Voorspel de uitkomst

Vraag 1

In een model staat een veel-op-veelrelatie tussen `Recept` en `Categorie`, en `Recept` heeft al attributen met algemene datatypes. NA en NNA staan er nog niet bij. Op welk niveau zit dit model?

Toon het antwoord ›

Vraag 2

Een ERD bevat `{NNA, DTR}` bij een foreign key, en `varchar(80)` als datatype. Op welk niveau zit het?

Toon het antwoord ›

Vraag 3

`Lid` is verbonden met `Beoordeling`, en `Beoordeling` met `Recept`. Iemand tekent ook een lijn tussen `Lid` en `Recept`, om te tonen welke recepten een lid beoordeeld heeft. Is die lijn nodig?

Toon het antwoord ›

Antwoorden

VRAAG 1

Logisch. Er zijn attributen en algemene datatypes, maar nog geen NA en NNA, en de veel-op-veelrelatie staat er nog. In een fysiek model mag dat niet meer.

‹ Terug naar de vraag

VRAAG 2

Fysiek. Delete-opties en NA of NNA horen enkel in een fysiek model, net als een datatype van een bepaald DBMS, zoals `varchar(80)`.

‹ Terug naar de vraag

VRAAG 3

Nee. Welke recepten een lid beoordeeld heeft, vind je al via `Beoordeling`. Een tweede pad is overbodig. Het kan je model zelfs tegenspreken: een lid dat via de nieuwe lijn aan een recept hangt, zonder beoordeling.

Heeft de lijn een andere betekenis, zoals *het lid dat het recept plaatste*, dan is het wel een eigen relatie. Zie hoofdstuk 6.

‹ Terug naar de vraag

3. Entiteiten en attributen

BOUWSTEEN

Wat het is

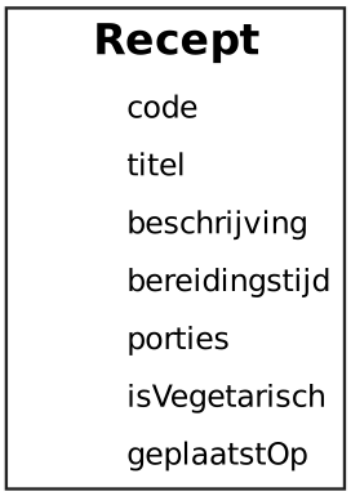
Een **entiteit** is iets waarover je gegevens wil bijhouden. Volgens de cursus is het:

- een persoon, een plaats, een ding of een gebeurtenis;
- reëel, zoals een lid, of abstract, zoals een beoordeling;
- van blijvend belang voor wat je modelleert;
- uniek identificeerbaar: je kan elke keer zeggen over welke het gaat.

Een entiteit wordt later een tabel. Voorbeelden bij Smaakvol: `Recept`, `Lid`, `Ingredient`, `Beoordeling`.

Een **attribuut** is een kenmerk van een entiteit. Het wordt later een kolom. Elk attribuut hoort bij precies één entiteit, en komt één keer voor per record: een recept heeft één titel, en die titel hoort bij dat ene recept.

Hoe het eruitziet



De entiteit Recept met haar attributen, in een logisch model.

De naam staat bovenaan, de attributen eronder. Datatypes en `{NNA}` komen er in het fysieke model bij.

Samengestelde gegevens

Een gegeven is **samengesteld** als het uit delen bestaat die je apart wil gebruiken. Bewaar zulke delen in aparte attributen.

Samengesteld	Splits je in	Waarom
naam	`voornaam`, `familienaam`	een mail die iemand aanspreekt met zijn voornaam. Uit *Jan Van den Broeck* haal je die achteraf niet betrouwbaar terug
adres	`straat`, `huisnummer`, `postcode`, `gemeente`	zoeken op gemeente, sorteren op postcode. Met enkel een postcode ben je trouwens niets: je hebt minstens straat en huisnummer nodig
bereidingstijd *1 u 30*	`bereidingstijd` in minuten: `90`	rekenen en sorteren gaat enkel met één getal

Wanneer is iets samengesteld? Dat hangt af van wat je ermee doet. De cursus geeft het voorbeeld *Bachelor in de Informatica*: als diploma is het één gegeven. Wil je kunnen zoeken op niveau, zoals graduaat of bachelor, dan zijn het er twee.

Procesgegevens

Een **procesgegeven** is een gegeven dat je kan berekenen uit andere gegevens. Bewaar het niet.

Het voorbeeld van de cursus is een leeftijd. Je bewaart de geboortedatum, en berekent de leeftijd als je ze nodig hebt. Een bewaarde leeftijd klopt na elke verjaardag niet meer, en moet elke dag voor elk record bijgewerkt worden.

Bij Smaakvol: de gemiddelde score van een recept, of het aantal beoordelingen. Die bereken je uit de beoordelingen zelf.

Een gegeven op de juiste plaats

Een attribuut hoort bij de entiteit die het beschrijft. Het voorbeeld uit de les: een verzekeringsagent krijgt een vaste commissie per type polis. Zet je dat percentage in elke polis, dan staat het er bij elke polis van dat type opnieuw, en verander je het op tien plaatsen tegelijk. Het hoort bij het type, niet bij de polis.

Bij Smaakvol: de naam van een keuken hoort niet in `Recept`. Anders staat *Italiaans* bij elk Italiaans recept opnieuw, en vroeg of laat ook als *italiaans* en *Italiaanse*. Zie hoofdstuk 9 voor de oplossing.

Waar je op let

CHECKLIST

- Elke entiteit is iets van blijvend belang, en uniek identificeerbaar.
- Elk attribuut beschrijft de entiteit waarin het staat, en komt één keer per record voor.
- Samengestelde gegevens zijn opgesplitst, als je de delen apart gebruikt.
- Er staan geen procesgegevens in.
- Een gegeven dat bij iets anders hoort, staat daar, en niet herhaald in elk record.

Veelgemaakte fouten

LET OP

- **Een rapport of een scherm als entiteit**, zoals `Overzicht` of `Zoekresultaat`. Dat zijn gegevens die je opvraagt, geen dingen die je bewaart.
- **Een attribuut dat meerdere waarden kan hebben**, zoals `ingredienten` in `Recept`. Dat wordt een eigen entiteit met een relatie.
- **Een percentage als tekst**, zoals `8%`. De cursus noteert `0.08`: een getal waarmee je kan rekenen.
- **Een berekend totaal bewaren**, zoals `aantalBeoordelingen`.

Uitgewerkt voorbeeld: van beschrijving naar entiteit

SITUATIE

Smaakvol organiseert kookworkshops. Een workshop heeft een code zoals *WS-0142*, een titel en een korte beschrijving. Ze vindt plaats op een datum, begint op een vast uur en duurt bijvoorbeeld *2 u 30*. Elke workshop heeft een eigen maximum aantal deelnemers, bijvoorbeeld 12, en op de website staat hoeveel plaatsen er nog vrij zijn. Elke workshop wordt gegeven door een chef. Bij de workshop staan de naam en het telefoonnummer van de chef.

Welke entiteit maak je, en welke attributen krijgt ze?

Stap 1. Is een workshop een entiteit? Toets aan de vier kenmerken.

Kenmerk	Workshop
een persoon, plaats, ding of gebeurtenis	✓ een gebeurtenis
reëel of abstract	✓ reëel: ze vindt echt plaats
van blijvend belang	✓ Smaakvol wil ze plannen, tonen en terugvinden
uniek identificeerbaar	✓ elke workshop heeft een eigen code

Het wordt een entiteit `Workshop`.

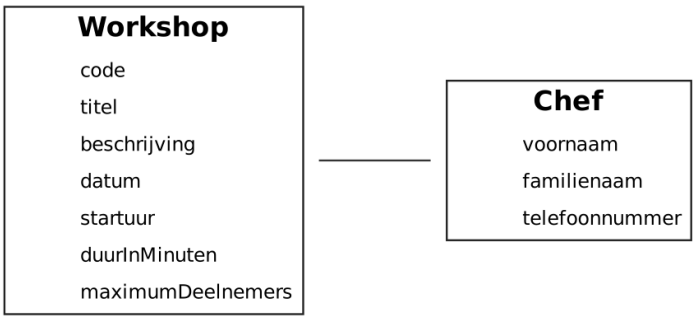
Stap 2. Welke gegevens noemt de beschrijving? Een code, een titel, een beschrijving, een datum, een startuur, een duur, een maximum aantal deelnemers, het aantal vrije plaatsen, en de naam en het telefoonnummer van de chef.

Stap 3. Elk gegeven toetsen. Niet alles wordt een attribuut van `Workshop`.

Gegeven	Beslissing	Waarom
code, titel, beschrijving, datum, startuur	✓ attribuut	één waarde per workshop, en het beschrijft de workshop
duur *2 u 30*	✓ attribuut, als één getal: `duurInMinuten`	een samengesteld gegeven. Met *2 u 30* kan je niet rekenen of sorteren, met `150` wel
maximum aantal deelnemers	✓ attribuut: `maximumDeelnemers`	één waarde per workshop
aantal vrije plaatsen	✗ niet bewaren	een procesgegeven: het maximum min het aantal inschrijvingen. Bewaar je het, dan klopt het na elke inschrijving niet meer
naam en telefoonnummer van de chef	✗ niet in `Workshop`	het beschrijft de chef, niet de workshop. Geeft dezelfde chef tien workshops, dan staat zijn nummer tien keer in de tabel. Het wordt een eigen entiteit `Chef`

Stap 4. De chef apart. `Chef` krijgt de gegevens die bij de chef horen. De naam is samengesteld: je splitst ze in `voornaam` en `familienaam`, zodat je een chef met zijn voornaam kan aanspreken.

Het resultaat. Een logisch model, nog zonder sleutels en datatypes. De lijn zegt dat workshop en chef samenhangen. Hoe dat precies gaat, staat in hoofdstuk 6.



Workshop en Chef, elk met haar eigen attributen.

Waarom zo.

- Een attribuut hoort bij de entiteit die het beschrijft, en komt één keer per record voor.
- Een samengesteld gegeven splits je, als je de delen apart gebruikt. Een duur bewaar je als één getal in één eenheid.
- Wat je kan berekenen, bewaar je niet.

Zo controleer je het zelf.

- Kan je bij elk attribuut zeggen: *één workshop heeft precies één …*? Lukt dat niet, dan hoort het er niet.
- Zou hetzelfde gegeven bij twee workshops opnieuw ingetypt worden? Dan hoort het bij iets anders.
- Kan je het gegeven uitrekenen uit wat er al is? Dan bewaar je het niet.

Voorspel de uitkomst

Vraag 1

Smaakvol wil leden een mail sturen die begint met *Dag* en hun voornaam. In `Lid` staat één attribuut `naam`, met waarden als *Sofie Van Damme* en *Jan Willem De Smet*. Lukt dat betrouwbaar?

Toon het antwoord ›

Vraag 2

`Recept` heeft een attribuut `aantalKeerBekeken`. Is dat een procesgegeven?

Toon het antwoord ›

Vraag 3

`Recept` heeft een attribuut `gemiddeldeScore`, en er is ook een entiteit `Beoordeling` met een score per lid. Wat is het probleem?

Toon het antwoord ›

Antwoorden

VRAAG 1

Nee. Bij *Jan Willem De Smet* weet je niet waar de voornaam stopt. Met aparte attributen `voornaam` en `familienaam` is het geen probleem.

‹ Terug naar de vraag

VRAAG 2

Nee. Je kan het niet berekenen uit andere gegevens in het model. Het is een teller die zelf bijgehouden wordt. Hield je elk bezoek bij in een eigen entiteit, dan was het wel een procesgegeven: dan tel je die bezoeken.

‹ Terug naar de vraag

VRAAG 3

Het is een procesgegeven. Je kan het gemiddelde berekenen uit de beoordelingen. Bewaar je het toch, dan klopt het niet meer zodra er een beoordeling bijkomt, verandert of verdwijnt, tenzij je het telkens bijwerkt.

‹ Terug naar de vraag

4. Naamgeving

BEGRIP

Wat het is

Een ERD wordt gelezen door anderen, en later wordt het een database waarin je queries schrijft. Vaste naamregels maken het leesbaar en voorspelbaar. Deze regels gelden in de cursus, en op het examen.

De regels

Wat	Regel	Goed	Fout
entiteit	begint met een hoofdletter, enkelvoud, zegt wat erin zit	`Recept`, `ReceptIngredient`	`recepten`, `Tbl_Recept`
attribuut	begint met een kleine letter, camelCase, enkelvoud, geen afkortingen, geen underscores	`bereidingstijd`, `publicatieDatum`	`Bereidingstijd`, `publicatie_datum`, `pubDat`
primaire sleutel	altijd `id`	`id`	`receptId`, `RECEPT_ID`
foreign key	de naam van de entiteit waarnaar ze verwijst, in camelCase, met `Id` erachter	`keukenId`, `lidId`	`keuken`, `fk_keuken`
relatie	een nummer: `R1`, `R2`, …	`R1`	
foreign key in het ERD	het nummer van haar relatie: `FK1` bij `R1`	`FK1`	

camelCase betekent: het eerste woord met een kleine letter, elk volgend woord met een hoofdletter. `commissiePercentage`, `aantalPorties`.

De naam van de entiteit in een attribuut

Herhaal de naam van de entiteit niet in haar attributen. In `Recept` heet het attribuut `titel`, niet `receptTitel`. In `Ingredient` heet het `naam`, niet `ingredientNaam`.

Enkel als het verwarring voorkomt, mag het. Het voorbeeld uit de les: in `Order` staat `orderdatum`, omdat *datum* alleen niet zegt of het om de besteldatum of de leverdatum gaat.

Eén taal, overal

Kies Nederlands of Engels, en houd je daaraan in het hele model. `Recept` met een attribuut `title` is niet consequent. De cursus zegt het zo: naamgeving moet consequent gebeuren, en hoe je dat doet, verschilt per opdrachtgever.

CHECKLIST

ERD-checklist uit les 1:

- Elke entiteit begint met een hoofdletter en staat in het enkelvoud.
- Elk attribuut begint met een kleine letter.
- De namen van de attributen staan in camelCase.

OPMERKING

Deze regels zijn een afspraak voor de cursus, geen wet. In bedrijven zie je ook `snake_case`, `PascalCase` voor kolommen, of `KlantId` als primaire sleutel. Welke stijl een bedrijf gebruikt, hangt af van dat bedrijf. Wat overal geldt: wees consequent.

In PostgreSQL worden namen zonder aanhalingstekens omgezet naar kleine letters. Een kolom `bereidingsTijd` heet dan `bereidingstijd`, tenzij je ze tussen dubbele aanhalingstekens zet. EF Core doet dat vanzelf. Zie hoofdstuk 34 van het C#-leerboek.

Veelgemaakte fouten

LET OP

- **Meervoud**: `Recepten` in plaats van `Recept`.
- **Afkortingen**: `aantPers` in plaats van `aantalPorties`.
- **Underscores**: `commissie_percentage`. Uit het voorbeeld van les 1.
- **Een PK met de naam van de tabel**: `receptId` als primaire sleutel van `Recept`. In de cursus is dat altijd `id`. `receptId` is de naam van een foreign key die naar `Recept` verwijst.
- **Twee talen door elkaar.**

Uitgewerkt voorbeeld: een ontwerp opkuisen

> **SITUATIE**
>
> Een collega tekende een eerste versie van de entiteit voor kookworkshops. Smaakvol wil ook een Engelse titel tonen voor anderstalige leden. Kijk elke naam na tegen de naamregels van de cursus.
>
> ```
> Workshops
>   WorkshopID
>   Workshop_Code
>   Titel
>   title_en
>   datum
>   startuur
>   duur_min
>   maxDeeln
>   ChefNaam
> ```

Stap 1. De naam van de entiteit. `Workshops` staat in het meervoud. De naam zegt wat één rij is: `Workshop`.

Stap 2. De primaire sleutel. `WorkshopID` volgt de regel niet. In de cursus heet de primaire sleutel altijd `id`.

Stap 3. Elk attribuut.

Naam	Wat er niet klopt	Regel	Na
`Workshop_Code`	de naam van de entiteit herhaald, een underscore, een hoofdletter	geen naam van de entiteit zonder reden, geen underscores, een kleine letter	`code`
`Titel`	een hoofdletter	een attribuut begint met een kleine letter	`titel`
`title_en`	een andere taal, een underscore	één taal in het hele model	`engelseTitel`
`datum`	✓ in orde	een workshop heeft maar één datum, dus er is geen verwarring	`datum`
`startuur`	✓ in orde		`startuur`
`duur_min`	een underscore, een afkorting	camelCase, geen afkortingen	`duurInMinuten`
`maxDeeln`	afkortingen	geen afkortingen	`maximumDeelnemers`
`ChefNaam`	een hoofdletter, en het hoort niet in deze entiteit	de naam van de chef beschrijft de chef	weg: wordt later een foreign key naar `Chef`

Het resultaat.



Workshop na de naamregels.

Waarom zo.

- Een entiteit: een hoofdletter, enkelvoud, een naam die zegt wat erin zit.
- Een attribuut: een kleine letter, camelCase, geen afkortingen, geen underscores.
- De naam van de entiteit enkel in een attribuut als het verwarring voorkomt. Bij `datum` is er geen verwarring. Had een workshop ook een inschrijvingsdeadline, dan zou je `datum` beter specifieker maken.
- Eén taal overal. Een Engelse titel is een gegeven, geen reden om de naam Engels te maken.

Zo controleer je het zelf. Loop naam per naam de ERD-checklist van les 1 af:

- Begint elke entiteit met een hoofdletter, in het enkelvoud?
- Begint elk attribuut met een kleine letter?
- Staan de namen van de attributen in camelCase?

En daarna de tabel *De regels* van dit hoofdstuk: geen underscores, geen afkortingen, en elke primaire sleutel heet `id`.

Voorspel de uitkomst

Vraag 1

Hoeveel fouten tegen de naamregels van de cursus zie je in deze entiteit?

```
Recepten
  receptId
  Titel
  bereidings_tijd
  aantPorties
```

Toon het antwoord ›

Vraag 2

`Beoordeling` verwijst naar `Lid` en naar `Recept`. Hoe heten de twee foreign keys?

Toon het antwoord ›

Vraag 3

In `Lid` staan de attributen `lidVoornaam`, `lidFamilienaam` en `lidEmail`. Wat zegt de cursus?

Toon het antwoord ›

Antwoorden

VRAAG 1

Vijf. `Recepten` staat in het meervoud: `Recept`. De primaire sleutel heet `id`. `Titel` begint met een hoofdletter. `bereidings_tijd` gebruikt een underscore: `bereidingstijd`. `aantPorties` is een afkorting: `aantalPorties`, of gewoon `porties`.

‹ Terug naar de vraag

VRAAG 2

`lidId` en `receptId`. De naam van de entiteit waarnaar ze verwijzen, in camelCase, met `Id` erachter.

‹ Terug naar de vraag

VRAAG 3

Laat `lid` weg: `voornaam`, `familienaam`, `email`. De naam van de entiteit hoort enkel in een attribuut als het verwarring voorkomt.

‹ Terug naar de vraag

5. Sleutels

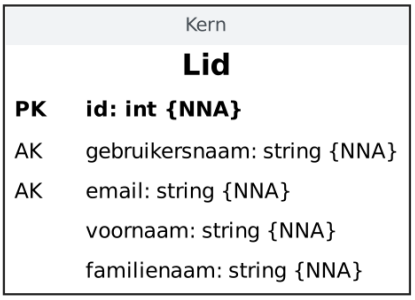
BOUWSTEEN

Wat het is

Een **sleutel** is een kolom, of een combinatie van kolommen, die een rij uniek identificeert. De cursus onderscheidt vier soorten.

Soort	Wat het is	Voorbeeld bij Smaakvol
primaire sleutel (PK)	de sleutel waarmee de tabel haar rijen identificeert. Komt nooit twee keer voor, en is nooit leeg	`id`
kandidaatsleutel	elk attribuut dat de primaire sleutel zou kunnen zijn	`code` van een recept, `gebruikersnaam` van een lid
natuurlijke sleutel	een kandidaatsleutel die uit de werkelijkheid komt	een factuurnummer, een productcode, een personeelsnummer
surrogaatsleutel	een sleutel zonder betekenis, enkel gemaakt om rijen te nummeren	`id`
alternatieve sleutel (AK)	een kandidaatsleutel die je niet als PK gebruikt	`code`, `email`

Hoe het eruitziet



Lid met een primaire sleutel en twee alternatieve sleutels.

`PK` staat voor de primaire sleutel, `AK` voor elke alternatieve sleutel. Twee aparte alternatieve sleutels krijgen elk hun eigen `AK`.

Waarom de PK altijd een surrogaatsleutel is

In de cursus is de primaire sleutel altijd `id`: een geheel getal zonder betekenis. De vier redenen uit de les:

Eigenschap	Waarom
geen betekenis	iemand typt een verkeerd rijksregisternummer in. De volgende dag wil de echte eigenaar van dat nummer zich inschrijven, en dat kan niet, want het bestaat al. Met een `id` zonder betekenis gebeurt dat niet
minimaal	hoe kleiner de sleutel, hoe sneller tabellen aan elkaar gekoppeld worden, en hoe minder schijfruimte
wordt nooit gewijzigd	een natuurlijke sleutel kan veranderen. Dan moet elke foreign key die ernaar verwijst mee veranderen. Een `id` verandert nooit
een geheel getal	getallen vergelijken gaat veel sneller dan teksten vergelijken

Een combinatie van alle kolommen is ook uniek, maar veel te traag. Die gebruik je nooit als primaire sleutel.

Alternatieve sleutels

Een kandidaatsleutel die je niet als primaire sleutel kiest, wordt een **alternatieve sleutel**. Ze blijft uniek: twee recepten mogen niet dezelfde `code` hebben.

- In SQL krijgt een alternatieve sleutel meestal een `UNIQUE`-constraint. Zie deel 2.
- Meestal is het een goed idee om er een **index** op te zetten. Een index laat de database snel zoeken op die kolom. Maar een index bouwen kost tijd, en bij elke wijziging moet hij bijgewerkt worden. Zet er dus enkel een waar het echt helpt.

Is een e-mailadres een goede kandidaatsleutel? Soms. Op een website waar het e-mailadres de gebruiker is, kan het. Maar e-mailadressen veranderen van eigenaar, en soms gebruiken meerdere mensen hetzelfde adres. Denk dus na of het in jouw situatie echt uniek is.

> **CHECKLIST**
>
> ERD-checklist uit les 2:
>
> - Alle kernentiteiten hebben minstens één alternatieve sleutel.

Veelgemaakte fouten

> **LET OP**
>
> - **Een natuurlijke sleutel als PK**, zoals `code` of een e-mailadres. Die wordt een AK, en `id` wordt de PK.
> - **Een kernentiteit zonder AK.** De checklist vraagt er minstens één.
> - **Een AK die niet echt uniek is**, zoals `titel` van een recept: twee leden kunnen een recept *Lasagne* plaatsen.
> - **Een PK van het type tekst.** In de cursus is de PK altijd een geheel getal.

Uitgewerkt voorbeeld: de sleutels van een kookboek

> **SITUATIE**
>
> Smaakvol houdt kookboeken bij waaruit recepten komen. Van elk kookboek kent Smaakvol de titel, de auteur, het uitgavejaar, de uitgeverij en het ISBN. Smaakvol neemt enkel boeken met een ISBN op. Van hetzelfde boek bestaan soms meerdere drukken, elk met een eigen ISBN.
>
> Welke sleutels krijgt `Kookboek`?

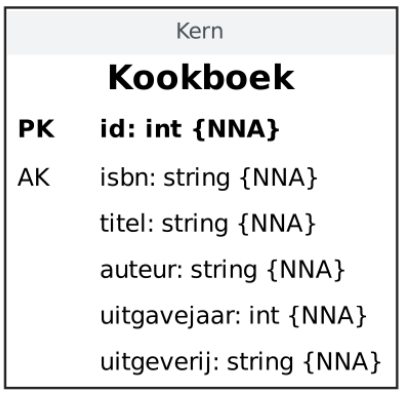
Stap 1. Zoek de kandidaatsleutels. Welke gegevens, alleen of samen, duiden altijd precies één kookboek aan?

Kandidaat	Uniek?	Waarom
`titel`	✗	een herziene druk heeft dezelfde titel
`titel` + `auteur`	✗	ook samen: een herziene druk heeft dezelfde titel en dezelfde auteur
`titel` + `auteur` + `uitgavejaar`	✗	twee drukken in hetzelfde jaar, of twee delen van hetzelfde boek, hebben ook die drie gelijk
`isbn`	✓	elke druk krijgt een eigen ISBN, en twee boeken delen er nooit een
`id`	✓	een getal zonder betekenis, dat je zelf toekent

Stap 2. Kies de primaire sleutel. In de cursus is dat altijd `id`. Waarom niet `isbn`? Het is een natuurlijke sleutel. Een tikfout in een ISBN moet je kunnen verbeteren, en dan zou elke foreign key die ernaar verwijst, mee moeten veranderen. Een `id` verandert nooit, en een getal vergelijken gaat sneller dan tekst.

Stap 3. De alternatieve sleutel. `isbn` blijft een kandidaatsleutel die je niet als PK kiest: het wordt een **alternatieve sleutel**, `AK`. In SQL krijgt ze later een `UNIQUE`-constraint.

Het resultaat.



Kookboek met een primaire sleutel en een alternatieve sleutel.

Waarom zo.

- De PK is altijd een surrogaatsleutel: `id`, een geheel getal zonder betekenis.
- Een natuurlijke sleutel wordt een AK.
- De ERD-checklist van les 2 vraagt minstens één AK bij elke kernentiteit.
- `isbn` is tekst, geen getal. Je rekent er niet mee. Zie hoofdstuk 9.

Zo controleer je het zelf.

- Kan je bij elke AK een voorbeeld bedenken van twee rijen met dezelfde waarde? Lukt dat, dan is het geen AK.
- Is de AK altijd ingevuld? Hier wel: Smaakvol neemt enkel boeken met een ISBN op.
- Heet de PK `id`, en is ze een getal?

Voorspel de uitkomst

Vraag 1

Smaakvol overweegt het e-mailadres van een lid als primaire sleutel. Een lid verandert van e-mailadres. Wat moet er dan allemaal gebeuren?

Toon het antwoord ›

Vraag 2

Welke van deze attributen van `Recept` is een goede alternatieve sleutel: `titel`, `code` of `publicatieDatum`?

Toon het antwoord ›

Vraag 3

`Lid` heeft de AK's `gebruikersnaam` en `email`. Hoeveel kandidaatsleutels heeft `Lid`?

Toon het antwoord ›

Antwoorden

VRAAG 1

Het e-mailadres moet in `Lid` veranderen, én in elke foreign key die ernaar verwijst: in elke beoordeling, elk recept en alles wat nog naar dat lid wijst. Met een `id` als PK verandert enkel de kolom `email` in die ene rij.

‹ Terug naar de vraag

VRAAG 2

`code`. Ze is per recept uniek. Twee recepten kunnen dezelfde titel hebben, en op dezelfde dag gepubliceerd worden.

‹ Terug naar de vraag

VRAAG 3

Drie: `id`, `gebruikersnaam` en `email`. Alle drie kunnen een lid uniek aanduiden. `id` is gekozen als PK, de andere twee zijn alternatieve sleutels.

‹ Terug naar de vraag

6. Relaties en foreign keys

BOUWSTEEN

Wat het is

Een **relatie** verbindt twee entiteiten. Je volgt ze om meer te weten over een record: van een recept naar zijn keuken, van een lid naar zijn beoordelingen.

Een relatie bestaat altijd uit een **foreign key** (FK), of refererende sleutel, in de ene tabel, die verwijst naar de **primaire sleutel** van de andere. Relaties zijn nodig om gegevens niet te herhalen: de naam van een keuken staat één keer in `Keuken`, en elk recept verwijst ernaar.

Er zijn drie soorten relaties.

Soort	Wat het betekent	Voorbeeld uit de les
één-op-één	één record aan elke kant	een persoon en zijn identiteitsbewijs
één-op-veel	één record aan de ene kant, meerdere aan de andere	een appartementsgebouw en zijn appartementen
veel-op-veel	meerdere aan beide kanten	vissers en wedstrijden

Eén-op-veel komt het meest voor. Eén-op-één is zeldzaam. Veel-op-veel los je op met een extra entiteit.

Waar staat de foreign key?

Een recept hoort bij één keuken, en een keuken heeft meerdere recepten. Waar komt de foreign key?

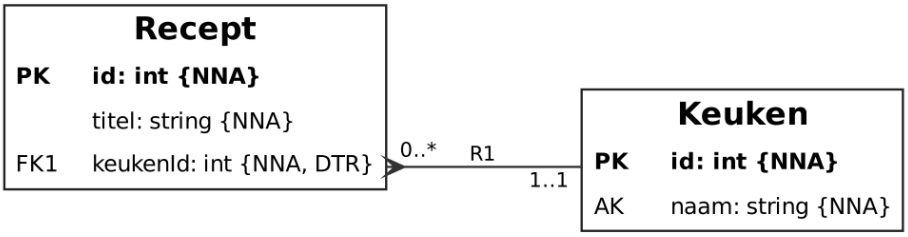
In `Keuken`: dan heeft `Keuken` een kolom `receptId`. Eén rij per recept, en de naam *Italiaans* staat er even vaak als er Italiaanse recepten zijn.

id	naam	receptId
1	Italiaans	1
2	Italiaans	4
3	Thais	2

In `Recept`: dan heeft `Recept` een kolom `keukenId`. Elke keuken staat één keer in `Keuken`.

id	titel	keukenId
1	Lasagne	1
2	Pad thai	2
4	Risotto	1

De tweede keuze is de goede. De foreign key staat altijd aan de **veel-kant**: de kant met de `*`.



De FK keukenId staat in Recept, de veel-kant van R1.

Een foreign key mag leeg zijn, als de relatie optioneel is: een recept zonder keuken heeft dan `NULL` in `keukenId`. Of dat mag, beslis je met de aard van de relatie. Zie hoofdstuk 7.

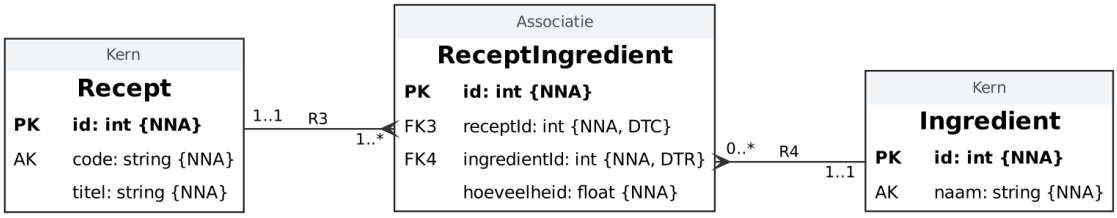
> **CHECKLIST**
>
> ERD-checklist uit les 2:
>
> - De relaties zijn opeenvolgend genummerd: `R1`, `R2`, … Het nummer heeft geen betekenis.
> - Elke FK heeft het nummer van haar relatie: `FK1` hoort bij `R1`.
> - De FK staat altijd aan de `*`-kant van een relatie.
> - Een FK bevat altijd de volledige PK van de entiteit aan de 1-kant.

Veel-op-veel: de associatie-entiteit

Een recept heeft meerdere ingrediënten, en een ingrediënt komt in meerdere recepten voor. Waar komt de foreign key nu?

- **In** `Ingredient`, als `receptId`: dan staat elk ingrediënt opnieuw in de tabel, voor elk recept waarin het zit.
- **In** `Recept`, als `ingredientId`: dan staat elk recept opnieuw in de tabel, voor elk ingrediënt.

Beide keren herhaal je gegevens. De oplossing is een **associatie-entiteit**: een tussentabel met een foreign key naar elk van de twee. De veel-op-veelrelatie wordt zo twee één-op-veelrelaties.



Een veel-op-veelrelatie, opgelost met de associatie-entiteit ReceptIngredient.

id	receptId	ingredientId
1	1	1
2	1	2
3	2	1

Rij 1 en 2 zeggen: lasagne bevat ingrediënt 1 en 2. Rij 3: pad thai bevat ook ingrediënt 1.

In de cursus heeft een associatie-entiteit zoals elke entiteit haar eigen `id` als primaire sleutel, met de twee foreign keys erbij. Ze kan ook eigen attributen krijgen: wat bij de combinatie hoort, zoals de `hoeveelheid` van dat ingrediënt in dat recept.

> **BELANGRIJK**
>
> De projectopgave zegt het letterlijk: plaats je een foreign key in de foute tabel, of herken je een veel-op-veelrelatie niet, dan heeft dat zware gevolgen voor je score. Een veel-op-veelrelatie mag in een conceptueel of logisch model staan, nooit in een fysiek model.

Eén-op-één: meestal samenvoegen

Een lid heeft één profiel, en een profiel hoort bij één lid. Een ERD en elk DBMS kunnen dat tekenen en bewaren, maar het brengt niets bij: enkel een extra tabel en een extra koppeling. De cursus raadt aan de twee entiteiten samen te voegen. Houd je ze toch apart, dan heb je daar een reden voor nodig.

Veelgemaakte fouten

> **LET OP**
>
> - **De FK aan de 1-kant**, zoals `receptId` in `Keuken`.
> - **Een veel-op-veelrelatie zonder associatie-entiteit** in het fysieke model.
> - **Een FK die maar een deel van de PK bevat**, of een andere kolom dan de PK, zoals de `naam` van de keuken.
> - **Relaties en foreign keys niet genummerd**, of een FK met een ander nummer dan haar relatie.
> - **Een één-op-éénrelatie zonder reden.**

Uitgewerkt voorbeeld: waar komt de foreign key?

> **SITUATIE**
>
> Een recept op Smaakvol kan uit een kookboek komen. Een kookboek bevat meerdere recepten. Bij een recept kan je ook aanduiden welke allergenen erin zitten: gluten, ei, melk, noten, … Een allergeen komt in veel recepten voor.
>
> Welke relaties zijn er, en waar staan de foreign keys?

Stap 1. Kookboek en recept: welke soort relatie? Eén kookboek, meerdere recepten. Eén recept, hoogstens één kookboek. Dat is **één-op-veel**.

Stap 2. Waar komt die foreign key? Probeer de twee plaatsen.

In `Kookboek`, als `receptId`: dan staat elk kookboek opnieuw in de tabel, voor elk recept dat eruit komt.

id	titel	receptId
1	Soepen voor elk seizoen	4
2	Soepen voor elk seizoen	9

In `Recept`, als `kookboekId`: elk kookboek staat één keer in `Kookboek`, en elk recept zegt uit welk boek het komt.

id	titel	kookboekId
1	Lasagne	NULL
4	Pompoensoep	1
9	Erwtensoep	1

De tweede is de goede: de foreign key staat aan de **veel-kant**, in `Recept`. Een recept dat niet uit een boek komt, heeft `NULL`.

Stap 3. Recept en allergeen: welke soort relatie? Een recept bevat meerdere allergenen, en een allergeen komt in meerdere recepten voor. Dat is **veel-op-veel**. Zet je `allergeenId` in `Recept`, dan staat elk recept opnieuw in de tabel, voor elk allergeen. Zet je `receptId` in `Allergeen`, dan staat elk allergeen opnieuw, voor elk recept. Beide keren herhaal je gegevens.

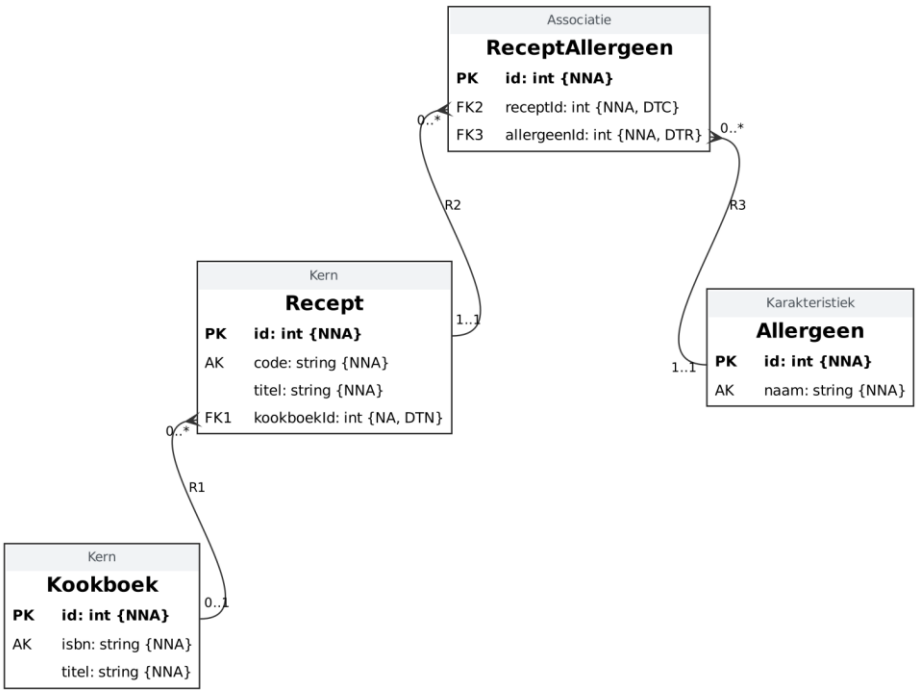
Stap 4. De associatie-entiteit. De veel-op-veelrelatie wordt een tussentabel `ReceptAllergeen`, met een foreign key naar elk van de twee:

id	receptId	allergeenId
1	1	1
2	1	2
3	4	3

Rij 1 en 2 zeggen: lasagne bevat allergeen 1 en 2. Rij 3: pompoensoep bevat allergeen 3.

Stap 5. Nummeren. In dit voorbeeld: `R1` tussen `Kookboek` en `Recept`, met `FK1` in `Recept`. `R2` en `R3` zijn de twee relaties van `ReceptAllergeen`, met `FK2` en `FK3`.

Het resultaat.



De foreign keys staan aan de veel-kant: in Recept en in ReceptAllergeen.

De multipliciteiten bij de lijnen leer je bepalen in hoofdstuk 7, de accolades in hoofdstuk 10 en 11.

Waarom zo.

- De foreign key staat altijd aan de `*`-kant, en bevat de volledige PK van de entiteit aan de 1-kant.
- Een veel-op-veelrelatie wordt in het fysieke model een associatie-entiteit met twee één-op-veelrelaties.
- Elke foreign key krijgt het nummer van haar relatie.

Zo controleer je het zelf.

- Schrijf voor elke relatie twee of drie rijen op, zoals hierboven. Moet je ergens een naam of een titel herhalen? Dan staat de foreign key aan de verkeerde kant, of mis je een associatie-entiteit.
- Staat bij elke foreign key het nummer van een lijn?

Voorspel de uitkomst

Vraag 1

Een lid kan meerdere recepten plaatsen. Elk recept is geplaatst door één lid. In welke entiteit staat de foreign key, en hoe heet ze?

Toon het antwoord ›

Vraag 2

Een recept kan in meerdere categorieën staan, zoals *Hoofdgerecht* en *Vegetarisch*, en een categorie bevat meerdere recepten. Hoeveel entiteiten heb je nodig in het fysieke model, en hoeveel foreign keys?

Toon het antwoord ›

Vraag 3

De relatie tussen `Recept` en `Keuken` heeft nummer `R1`. De foreign key `keukenId` staat in `Recept`. Hoe heet ze in het ERD?

Toon het antwoord ›

Antwoorden

VRAAG 1

In `Recept`, als `lidId`. `Recept` is de veel-kant: één lid, meerdere recepten.

Een naam die de rol beschrijft, zoals `auteurId`, mag ook, als dat duidelijker is. Dat zie je vooral als twee foreign keys naar dezelfde entiteit verwijzen, zoals in hoofdstuk 12.

‹ Terug naar de vraag

VRAAG 2

Drie entiteiten en twee foreign keys: `Recept`, `Categorie` en een associatie-entiteit `ReceptCategorie`, met de foreign keys `receptId` en `categorieId`. Beide staan in de associatie-entiteit.

‹ Terug naar de vraag

VRAAG 3

`FK1`. Een foreign key krijgt het nummer van haar relatie.

‹ Terug naar de vraag

7. Aard, connectiviteit en relatiebeschrijving

BOUWSTEEN

Wat het is

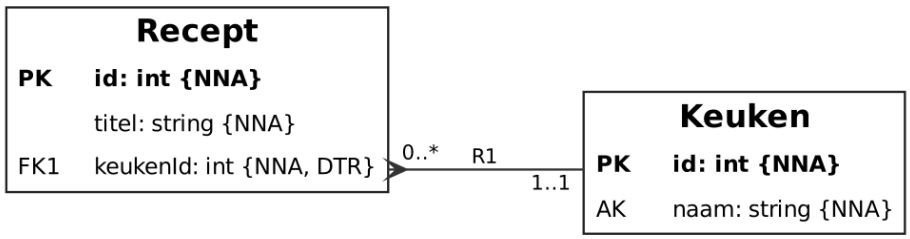
Aan elk uiteinde van een relatie staat een **multipliciteit**: twee getallen met twee puntjes ertussen, zoals `0..*`. Het eerste getal is de **aard**, het tweede de **connectiviteit**.

Notatie	Aard	Connectiviteit	Lees als
`0..1`	0: optioneel	1	nul of één
`1..1`	1: verplicht	1	precies één
`0..*`	0: optioneel	`*`: veel	nul of meer
`1..*`	1: verplicht	`*`: veel	één of meer

- De **aard** zegt of de relatie verplicht is: moet er aan de overkant iets zijn? Aard 0 is optioneel, aard 1 is verplicht.
- De **connectiviteit** zegt hoeveel: één, of veel.

Een relatie geldt in twee richtingen. Je bepaalt de multipliciteit dus aan beide kanten.

Hoe het eruitziet



R1 met de multipliciteit aan elke kant: 0.. bij Recept, 1..1 bij Keuken.*

Je leest een uiteinde vanuit de **overkant**. Het `0..*` bij `Recept` zegt: één keuken heeft nul of meer recepten. Het `1..1` bij `Keuken` zegt: één recept hoort bij precies één keuken.

Hoe je de aard bepaalt

Stel aan elke kant de vraag: **kan het ene bestaan zonder het andere?**

- Kan een keuken bestaan zonder recepten? Ja. Een nieuwe keuken heeft nog geen recepten. Aan de receptkant staat dus aard 0: `0..*`.
- Kan een recept bestaan zonder keuken? Bij Smaakvol niet: elk recept krijgt een keuken. Aan de keukenkant staat dus aard 1: `1..1`.

Dat zijn geen theoretische keuzes. Ze bepalen wat de database later toelaat: of `keukenId` leeg mag blijven, en of je een keuken kan verwijderen. Zie hoofdstuk 10 en hoofdstuk 11.

Er zijn drie combinaties van aard: optioneel aan beide kanten, optioneel aan één kant en verplicht aan de andere, of verplicht aan beide kanten.

Hoe je de connectiviteit bepaalt

Stel aan elke kant de vraag: **hoeveel van het ene horen bij één van het andere?**

- Hoeveel recepten heeft één keuken? Veel: `*`.
- Hoeveel keukens heeft één recept? Eén: `1`.

Neem de antwoorden uit wat je modelleert, niet uit wat makkelijk tekent.

De relatiebeschrijving

Elke relatie krijgt een beschrijving in gewone zinnen. Zo begrijpt iedereen die het model leest, wat de lijn betekent. De cursus schrijft ze zo:

> ***R1**: Een keuken heeft geen, één of meerdere recepten. Een recept hoort bij precies één keuken.*

- Beschrijf van de linkse entiteit naar de rechtse, of van de bovenste naar de onderste.
- Kan dat niet, of kan er verwarring zijn, zet dan een pijl bij de relatie.
- Een veel-op-veelrelatie mag je in één keer beschrijven: *Een recept bevat meerdere ingrediënten, en een ingrediënt komt in meerdere recepten voor.*

Elk woord van de beschrijving moet kloppen met de multipliciteit. *Geen, één of meerdere* hoort bij `0..*`. *Precies één* hoort bij `1..1`. *Minstens één* hoort bij `1..*`.

Veelgemaakte fouten

LET OP

- **De multipliciteit aan de verkeerde kant.** Het `*` voor *meerdere recepten* staat aan de kant van `Recept`.
- **Enkel één kant invullen.** Een relatie heeft altijd twee multipliciteiten.
- **Een beschrijving die iets anders zegt dan de multipliciteit**, zoals *een recept heeft altijd een keuken* bij `0..1`.
- **Aard 1 aan beide kanten zonder dat het kan.** Moet een keuken minstens één recept hebben, en een recept een keuken, dan kan je geen nieuwe keuken toevoegen zonder tegelijk een recept.

Uitgewerkt voorbeeld: multipliciteiten en beschrijvingen

SITUATIE

Smaakvol heeft drie relaties: `R1` tussen `Kookboek` en `Recept`, `R2` tussen `Recept` en de associatie-entiteit `ReceptAllergeen`, en `R3` tussen `Allergeen` en `ReceptAllergeen`. Een rij van `ReceptAllergeen` noemen we hier een **vermelding**: één allergeen in één recept. Wat Smaakvol erover zegt:

- De meeste recepten schrijven leden zelf. Ze komen niet uit een kookboek.
- Een kookboek kan in Smaakvol staan voor er een recept uit gekozen is.
- Een recept zonder allergenen kan.
- Een allergeen kan in de lijst staan zonder dat een recept het al bevat.

Bepaal de multipliciteiten, en schrijf de relatiebeschrijvingen.

Stap 1. R1, aan de kant van `Kookboek`. Je leest vanuit `Recept`.

- Aard: kan een recept bestaan zonder kookboek? Ja, de meeste. Aard **0**.
- Connectiviteit: uit hoeveel kookboeken komt één recept? Eén. Connectiviteit **1**.

Aan de kant van `Kookboek` staat `0..1`.

Stap 2. R1, aan de kant van `Recept`. Je leest vanuit `Kookboek`.

- Aard: kan een kookboek bestaan zonder recepten? Ja. Aard **0**.
- Connectiviteit: hoeveel recepten heeft één kookboek? Veel. `*`.

Aan de kant van `Recept` staat `0..*`.

Stap 3. R2 en R3. Een rij in `ReceptAllergeen` hoort altijd bij precies één recept en één allergeen: `1..1` aan de kant van `Recept` en van `Allergeen`. Een recept kan geen, één of meerdere rijen hebben, en een allergeen ook: `0..*` aan de kant van `ReceptAllergeen`, bij allebei.

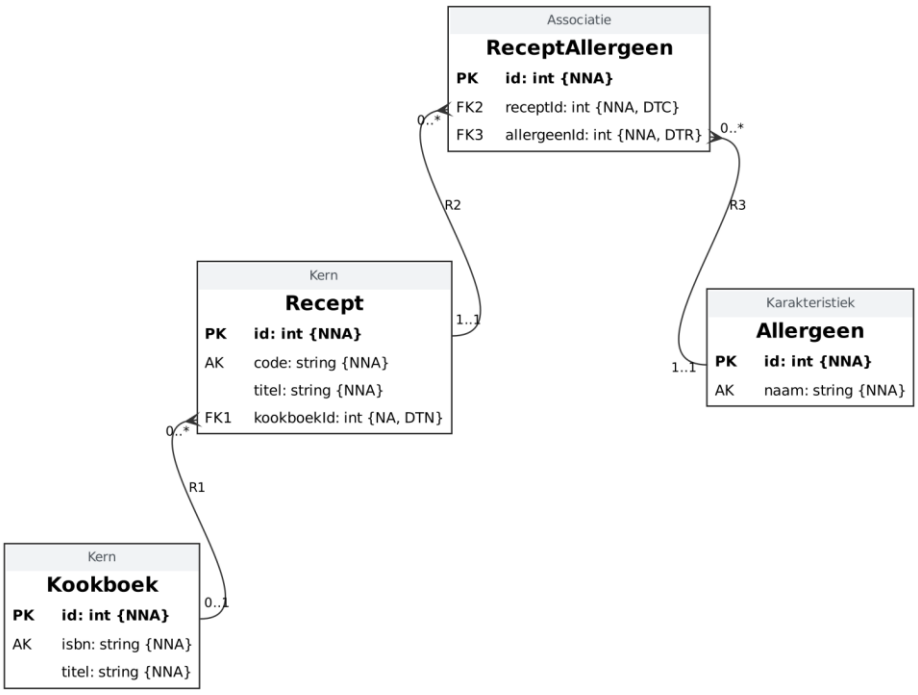
Relatie	Kant	Aard	Connectiviteit	Multipliciteit
R1	`Kookboek`	0: een recept kan zonder boek	1	`0..1`
R1	`Recept`	0: een boek kan zonder recepten	`*`	`0..*`
R2	`Recept`	1: elke rij hoort bij een recept	1	`1..1`
R2	`ReceptAllergeen`	0: een recept kan zonder allergenen	`*`	`0..*`
R3	`Allergeen`	1: elke rij hoort bij een allergeen	1	`1..1`
R3	`ReceptAllergeen`	0: een allergeen kan zonder recepten	`*`	`0..*`

Stap 4. De relatiebeschrijvingen. Elk woord moet kloppen met de multipliciteit. Je beschrijft van boven naar onder, zoals de entiteiten in het ERD hieronder staan.

> ***R1**: Een recept komt uit geen of één kookboek. Een kookboek bevat geen, één of meerdere recepten.*
>
> ***R2**: Een vermelding hoort bij precies één recept. Een recept heeft geen, één of meerdere vermeldingen.*
>
> ***R3**: Een vermelding hoort bij precies één allergeen. Een allergeen heeft geen, één of meerdere vermeldingen.*

Wil je de veel-op-veelrelatie in één keer beschrijven, dan mag dat ook: *Een recept bevat geen, één of meerdere allergenen, en een allergeen komt in geen, één of meerdere recepten voor.*

Het resultaat.



De drie relaties met hun multipliciteiten.

Waarom zo.

- Een multipliciteit lees je vanuit de overkant. Het `0..1` bij `Kookboek` zegt iets over één recept.
- Aard 0 is optioneel, aard 1 verplicht. De aard aan de 1-kant bepaalt later of de foreign key NA of NNA is: `kookboekId` wordt NA. Zie hoofdstuk 10.
- *Geen, één of meerdere* hoort bij `0..*`, *precies één* bij `1..1`, *geen of één* bij `0..1`.

Zo controleer je het zelf.

- Lees elke multipliciteit hardop als zin, vanuit de overkant. Klopt de zin met wat Smaakvol zei?
- Staat de `*` aan de kant van de entiteit met de foreign key?
- Spreekt elk woord van je beschrijving de multipliciteit niet tegen?

Voorspel de uitkomst

Vraag 1

Een beoordeling is altijd van één lid. Een lid kan geen, één of meerdere beoordelingen geven. Welke multipliciteit staat aan de kant van `Beoordeling`, en welke aan de kant van `Lid`?

Toon het antwoord ›

Vraag 2

Bij de relatie tussen `Recept` en `Foto` staat aan de kant van `Recept` `0..1`, en aan de kant van `Foto` `0..*`. Schrijf de relatiebeschrijving.

Toon het antwoord ›

Vraag 3

Aan de kant van `ReceptIngredient` staat `1..*`, aan de kant van `Recept` `1..1`. Kan je een recept bewaren zonder ingrediënten?

Toon het antwoord ›

Antwoorden

VRAAG 1

`0..*` bij `Beoordeling`, `1..1` bij `Lid`. Eén lid heeft nul of meer beoordelingen. Eén beoordeling hoort bij precies één lid.

‹ Terug naar de vraag

VRAAG 2

Een recept heeft geen, één of meerdere foto's. Een foto hoort bij geen of één recept.

`0..1` bij `Recept` betekent dat een foto ook zonder recept kan bestaan, bijvoorbeeld een foto die nog niet gebruikt wordt.

‹ Terug naar de vraag

VRAAG 3

Nee, niet volgens het model. `1..*` zegt: elk recept heeft minstens één ingrediëntregel. Een recept zonder ingrediënten is volgens het model ongeldig.

Een database kan *minstens één* niet afdwingen met een foreign key. Dat moet je software bewaken. Wat de database wel afdwingt, is de andere kant: elke regel hoort bij een bestaand recept.

‹ Terug naar de vraag

8. Soorten entiteiten

BOUWSTEEN

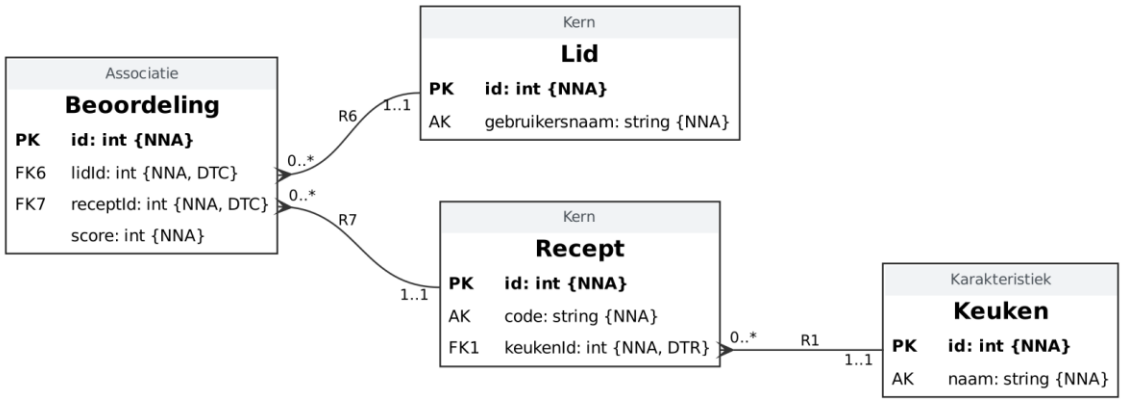
Wat het is

De cursus onderscheidt drie soorten entiteiten.

Soort	Wat het is	Voorbeelden bij Smaakvol
kernentiteit	onafhankelijk: ze bestaat op zichzelf en heeft op zichzelf betekenis. Ze is datgene waarover je toepassing gaat	`Recept`, `Lid`, `Ingredient`
karakteristieke entiteit	bestaat niet op zichzelf. Ze dient enkel om een andere entiteit te verduidelijken, te kwalificeren of te karakteriseren	`Keuken`, `Categorie`, `Eenheid`
associatie-entiteit	stelt een veel-op-veelrelatie voor, tussen twee of meer entiteiten	`ReceptIngredient`, `Beoordeling`

Hoe het eruitziet

Het voorbeeld uit de les: een `Klant` (kern), een `Communicatie` met enkel een omschrijving zoals *e-mail* of *telefoon* (karakteristiek), en `KlantCommunicatie`, die zegt welke klant via welk kanaal bereikbaar is (associatie).



De drie soorten bij Smaakvol, elk met haar soort boven de naam.

De projectopgave vraagt de **benoeming van de soort** van elke entiteit. In de voorbeeldmodellen van de cursus staat de soort boven de naam van elke entiteit.

Hoe je de soort herkent

Vraag	Ja	Nee
Heeft ze twee foreign keys, en bestaat ze om twee andere entiteiten aan elkaar te koppelen?	associatie	ga verder
Bestaat ze enkel om een andere entiteit te beschrijven, met weinig attributen, zoals een keuzelijst?	karakteristiek	ga verder
Gaat je toepassing erover, en bestaat ze op zichzelf?	kern	kijk opnieuw

Een karakteristieke entiteit heeft meestal weinig attributen, vaak enkel een naam, en weinig relaties. De lijst kan wel groeien: er komt al eens een keuken bij. Zie hoofdstuk 9.

Een associatie-entiteit kan eigen attributen hebben. `Beoordeling` koppelt een lid aan een recept, maar heeft ook een `score`, een `commentaar` en een `datum`. Het blijft een associatie-entiteit: elke beoordeling is één combinatie van één lid en één recept.

Veelgemaakte fouten

> **LET OP**
>
> - **De soort niet vermelden.** De opgave vraagt ze bij elke entiteit.
> - **Een karakteristieke entiteit als kern aanduiden**, omdat ze een eigen tabel heeft. Een eigen tabel maakt nog geen kern: de vraag is of ze op zichzelf betekenis heeft.
> - **Een associatie-entiteit niet herkennen**, omdat ze een eigen naam heeft, zoals `Beoordeling` of `Bestelling`.

Uitgewerkt voorbeeld: de soort van elke entiteit

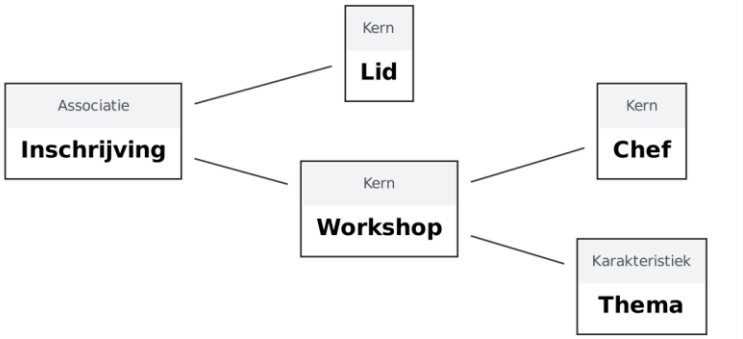
> **SITUATIE**
>
> Het deel van Smaakvol voor kookworkshops heeft vijf entiteiten:
>
> - `Lid`: een lid van Smaakvol, met een gebruikersnaam en een naam;
> - `Workshop`: een kookworkshop, met een code, een titel, een datum en een startuur;
> - `Chef`: wie de workshop geeft, met een naam en een telefoonnummer. Een chef geeft meerdere workshops;
> - `Inschrijving`: met `lidId`, `workshopId` en `ingeschrevenOp`;
> - `Thema`: met enkel een `naam`, zoals *Italiaans*, *Bakken* of *Vegan*. Er komen geregeld thema's bij.
>
> Benoem de soort van elke entiteit.

Stap 1. Stel de drie vragen, in volgorde. De eerste vraag waarop je *ja* antwoordt, geeft de soort.

Entiteit	Koppelt ze twee entiteiten, met twee foreign keys?	Beschrijft ze enkel een andere entiteit?	Bestaat ze op zichzelf?	Soort
`Inschrijving`	✓ ja: een lid en een workshop			associatie
`Thema`	✗	✓ ja: een keuzelijst die een workshop kwalificeert		karakteristiek
`Lid`	✗	✗	✓	kern
`Workshop`	✗	✗	✓	kern
`Chef`	✗	✗: een chef is een persoon, met eigen gegevens	✓	kern

Stap 2. Twijfelgevallen. `Chef` heeft maar een paar attributen. Wordt ze daardoor karakteristiek? Nee: de vraag is of ze op zichzelf betekenis heeft. Een chef is een persoon van blijvend belang, die je ook zonder workshop wil kennen. `Inschrijving` heeft een eigen naam en een eigen attribuut, maar blijft één combinatie van één lid en één workshop: een associatie-entiteit.

Het resultaat. De soort staat boven de naam van elke entiteit, zoals in de voorbeeldmodellen van de cursus.



Vijf entiteiten, elk met haar soort.

Waarom zo.

- Kern: onafhankelijk, met betekenis op zichzelf. Het is waar de toepassing over gaat.
- Karakteristiek: bestaat enkel om een andere entiteit te verduidelijken, vaak een keuzelijst.
- Associatie: stelt een veel-op-veelrelatie voor, ook als ze een eigen naam of eigen attributen heeft.
- De soort volgt uit de betekenis, niet uit het aantal attributen of uit waar de foreign key staat.

Zo controleer je het zelf.

- Kan je bij elke associatie-entiteit de zin schrijven *een … kan bij meerdere … horen, en een … bij meerdere …*?
- Zou een karakteristieke entiteit nog iets betekenen zonder de entiteit die ze beschrijft? Zo ja, kijk opnieuw.
- Staat de soort bij elke entiteit, en op één plaats?

Voorspel de uitkomst

Vraag 1

`Moeilijkheidsgraad` heeft enkel `id` en `naam`, met waarden als *makkelijk* en *gemiddeld*. Elk recept verwijst ernaar. Welke soort is het?

Toon het antwoord ›

Vraag 2

`Favoriet` heeft `id`, `lidId`, `receptId` en `toegevoegdOp`. Welke soort is het?

Toon het antwoord ›

Vraag 3

`Lid` heeft geen enkele foreign key, en `Recept` heeft er een naar `Lid`. Wordt `Recept` daardoor een karakteristieke entiteit van `Lid`?

Toon het antwoord ›

Antwoorden

VRAAG 1

Karakteristiek. Ze bestaat enkel om een recept te kwalificeren. Een moeilijkheidsgraad zonder recepten zegt niets.

Een lijst die zo klein en vast is, kan ook een enumeratie zijn in plaats van een entiteit. Zie hoofdstuk 9.

‹ Terug naar de vraag

VRAAG 2

Associatie. Ze koppelt leden aan recepten in een veel-op-veelrelatie: een lid heeft meerdere favorieten, en een recept is de favoriet van meerdere leden. `toegevoegdOp` is een attribuut van die koppeling.

‹ Terug naar de vraag

VRAAG 3

Nee. Een recept heeft op zichzelf betekenis: het is waar Smaakvol om draait. Dat het naar een lid verwijst, maakt het niet karakteristiek. De soort volgt uit de betekenis, niet uit waar de foreign key staat.

‹ Terug naar de vraag

9. Datatypes en zelfgemaakte types

BOUWSTEEN

Wat het is

Het **datatype** van een attribuut bepaalt drie dingen:

- welke waarden het kan aannemen;
- hoe die waarden bewaard worden;
- welke bewerkingen je erop kan doen: rekenen, sorteren, tekst knippen.

Je noteert het datatype achter de naam van het attribuut, na een dubbelpunt: `porties: int`.

Logisch en fysiek

Welk datatype je schrijft, hangt af van het niveau van je model.

Niveau	Datatypes	Voorbeelden
logisch	algemene types, voor elk DBMS dezelfde	`int`, `float`, `string`, `date`, `time`, `datetime`, `boolean`
fysiek, SQL Server	de types van SQL Server	`int`, `numeric`, `varchar`, `datetime2`, …
fysiek, MySQL	de types van MySQL	`int`, `float`, `double`, `varchar`, `datetime`, …
fysiek, Oracle	de types van Oracle	`number`, `varchar2`, `date`, …

De voorbeeldmodellen van de cursus gebruiken beide: soms `string`, soms `varchar(50)`. De datatypes van SQL Server komen uitgebreid in deel 2.

Algemeen type	Waarvoor	Voorbeeld bij Smaakvol
`int`	gehele getallen waarmee je rekent of telt	`porties`, `bereidingstijd` in minuten
`float`	kommagetallen	`hoeveelheid`, `prijs`
`string`	tekst	`titel`, `postcode`, `telefoonnummer`
`date`	een datum zonder uur	`geboortedatum`
`time`	een uur zonder datum	`startuur` van een kookworkshop
`datetime`	een datum met uur	`geplaatstOp`
`boolean`	ja of nee	`isVegetarisch`

Enkel een getal als je ermee rekent

De vuistregel van de cursus: gebruik enkel een getaltype als je met de waarde wil rekenen.

Gegeven	Type	Waarom
telefoonnummer	`string`	je rekent er nooit mee. En als getal verlies je de 0 vooraan: `0472…` wordt `472…`
huisnummer	`string`	`12B` is ook een huisnummer
postcode	`string`	je rekent er niet mee, en in andere landen bevat ze letters
salaris	`float`	je telt, vergelijkt en rekent ermee
porties	`int`	je deelt of vermenigvuldigt ermee om een recept om te rekenen

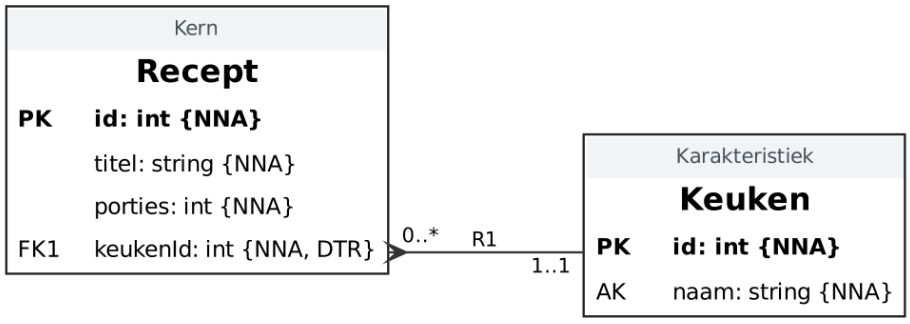
Een eenheid hoort niet in de waarde. Schrijf `bereidingstijd: int` met de waarde `90`, niet `string` met *90 min*. Wil je de eenheid tonen, dan zet je ze in de naam, of in een eigen attribuut als ze kan verschillen.

Zelfgemaakte types

Soms komt een waarde steeds terug uit een beperkte lijst: de keuken van een recept, de moeilijkheid, de eenheid van een ingrediënt. Laat je dat vrij intypen, dan krijg je *Italiaans*, *italiaans*, *Italiaanse* en *Italie* door elkaar. Het voorbeeld uit de les zijn automerken: *VW*, *vW*, *volkswagen*, *volksw.*, *UW*.

Je wil dat zo'n waarde overal hetzelfde ingevuld wordt, en dat de gebruiker ze uit een keuzelijst kiest in plaats van ze te typen. Daarvoor heb je twee mogelijkheden.

Een karakteristieke entiteit. Je maakt een aparte entiteit met de mogelijke waarden, en de andere entiteit verwijst ernaar met een foreign key.

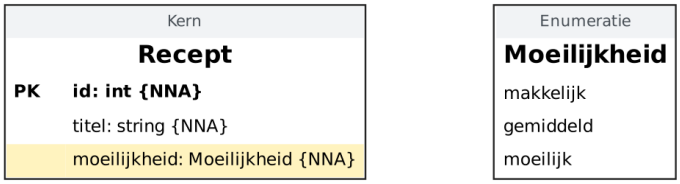


Keuken als karakteristieke entiteit.

> ***R1**: Een recept hoort bij precies één keuken. Een keuken kan bij meerdere recepten horen.*

- weinig attributen, vaak enkel een naam;
- weinig relaties met andere entiteiten;
- de lijst ligt niet vast: er kan een keuken bijkomen.

Een enumeratie (enum). Je noteert de mogelijke waarden in het model zelf, zonder aparte entiteit.



Moeilijkheid als enumeratie: een vaste lijst bij het attribuut.

- een vaste, kleine lijst waarden;
- geen aparte entiteit, en dus geen foreign key;
- je kan er geen extra eigenschappen aan hangen.

In draw.io teken je een enumeratie met de vorm *List*, en zet je de mogelijke waarden eronder.

Bij het attribuut schrijft dit boek de naam van de enumeratie als datatype: `moeilijkheid: Moeilijkheid {NNA}`. De dia's tonen het attribuut naast de lijst, maar leggen niet vast welk datatype je erbij noteert. Vraag na hoe je leerkracht het verwacht.

Hoe je kiest

> **BELANGRIJK**
>
> **Vuistregel uit les 3:** gebruik een enumeratie als de lijst klein en stabiel is. Gebruik een karakteristieke entiteit als de waarden vaak wijzigen, of als je er extra gegevens of relaties aan wil koppelen.

Lijst	Keuze	Waarom
moeilijkheid: makkelijk, gemiddeld, moeilijk	enum	klein en vast
keuken: Italiaans, Thais, Belgisch, …	karakteristieke entiteit	er komen keukens bij
eenheid: gram, liter, stuk, eetlepel, …	karakteristieke entiteit	er komen eenheden bij, en misschien wil je een afkorting bij elke eenheid
dag van de week	enum	ligt helemaal vast

Veelgemaakte fouten

LET OP

- **Een getaltype voor een telefoonnummer, postcode of huisnummer.**
- **Een eenheid in de waarde**, zoals *90 min* of *250 g*.
- **Een percentage als tekst met een procentteken.** Schrijf `0.08` als `float`.
- **Een vrij tekstveld voor een waarde uit een vaste lijst.** Gebruik een enum of een karakteristieke entiteit.
- **Een enum voor een lijst die groeit**, of waar je eigen gegevens aan wil hangen.
- **Datatypes van twee niveaus door elkaar**, zoals `string` en `varchar(50)` in hetzelfde model.

OPMERKING

In PostgreSQL bestaat een echt enum-type: `CREATE TYPE moeilijkheid AS ENUM (...)`. SQL Server heeft dat niet. Daar dwing je een vaste lijst af met een `CHECK`-constraint. Zie deel 2.

Uitgewerkt voorbeeld: datatypes en een vaste lijst

SITUATIE

Een workshop van Smaakvol heeft deze attributen: `code` (zoals *WS-0142*), `titel`, `beschrijving`, `datum`, `startuur`, `duurInMinuten`, `maximumDeelnemers` en `prijs` in euro. Een chef heeft ook een `telefoonnummer`.

Daarnaast heeft elke workshop een **niveau**: *beginner*, *gevorderd* of *expert*. Andere niveaus komen er niet. En een workshop kan een **thema** hebben, zoals *Italiaans*, *Bakken* of *Vegan*. Er komen geregeld thema's bij, en Smaakvol wil bij elk thema een korte beschrijving tonen.

Geef elk attribuut een logisch datatype, en beslis hoe je niveau en thema modelleert.

Stap 1. Elk attribuut een datatype. Gebruik de algemene types van een logisch model, en enkel een getal als je ermee rekent.

Attribuut	Datatype	Waarom
`code`	`string`	bevat letters en een streepje
`titel`, `beschrijving`	`string`	tekst
`datum`	`date`	een datum zonder uur
`startuur`	`time`	een uur zonder datum
`duurInMinuten`	`int`	je rekent ermee: het einduur is het startuur plus de duur
`maximumDeelnemers`	`int`	je vergelijkt het met het aantal inschrijvingen
`prijs`	`float`	een kommagetal: `24.50`
`telefoonnummer` van `Chef`	`string`	je rekent er nooit mee, en als getal verlies je de 0 vooraan

Stap 2. Niveau: een enumeratie of een karakteristieke entiteit? Gebruik de vuistregel uit les 3.

Vraag	Niveau
is de lijst klein?	✓ drie waarden
is ze stabiel?	✓ er komen geen niveaus bij
wil je er eigen gegevens of relaties aan hangen?	✗ nee
keuze	**enumeratie** `Niveau`

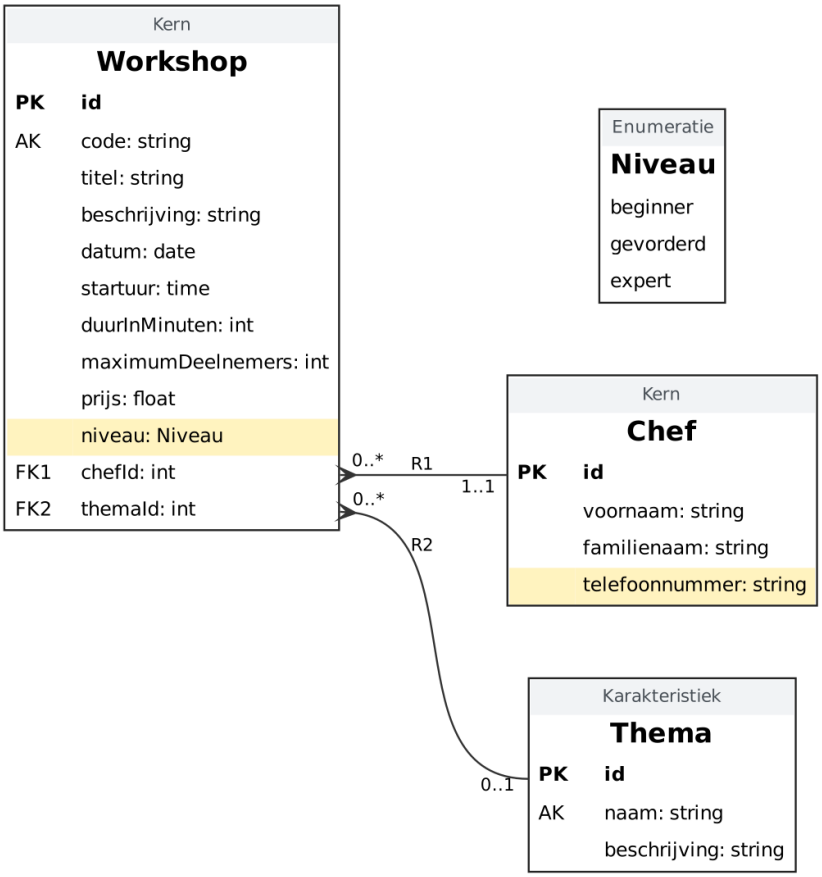
Het attribuut in `Workshop` wordt `niveau: Niveau`. Zo noteert dit boek het; de dia's leggen die notatie niet vast, dus vraag na hoe je leerkracht ze wil. De lijst teken je met de vorm *List*, met de waarden eronder.

Stap 3. Thema: dezelfde vragen.

Vraag	Thema
is de lijst klein?	½ nu wel
is ze stabiel?	✗ er komen geregeld thema's bij
wil je er eigen gegevens of relaties aan hangen?	✓ een beschrijving bij elk thema
keuze	**karakteristieke entiteit** `Thema`

`Workshop` verwijst naar `Thema` met een foreign key `themaId`.

Het resultaat.



Workshop met datatypes, de enumeratie Niveau en de karakteristieke entiteit Thema.

Waarom zo.

- Datatypes van één niveau: hier de algemene types van een logisch model, niet gemengd met `varchar(50)`.
- Een eenheid hoort niet in de waarde: `duurInMinuten` met `150`, niet *2 u 30*. `prijs` met `24.50`, niet *€ 24,50*.
- Een enumeratie voor een kleine, vaste lijst. Een karakteristieke entiteit voor een lijst die groeit, of waar je gegevens aan wil hangen.

Zo controleer je het zelf.

- Bij elk getaltype: waarmee reken je?
- Bij elke enumeratie: kan er morgen een waarde bijkomen, of wil je er iets bij bewaren? Dan is het een entiteit.
- Gebruikt een attribuut je enumeratie? Een lijst die niemand gebruikt, zegt niets.

Voorspel de uitkomst

Vraag 1

Smaakvol bewaart het telefoonnummer *0472 12 34 56* in een attribuut `telefoonnummer: int`. Wat staat er in de database?

Toon het antwoord ›

Vraag 2

Smaakvol wil bij elke keuken ook het land van herkomst en een korte uitleg tonen. `keuken` is nu een enum. Wat moet er veranderen?

Toon het antwoord ›

Vraag 3

Welk algemeen datatype krijgt `isVegetarisch`, en welk `geplaatstOp`, het moment waarop een recept online kwam?

Toon het antwoord ›

Antwoorden

VRAAG 1

472123456. Een getal heeft geen 0 vooraan en geen spaties. Bovendien rekent niemand met een telefoonnummer. Het hoort een `string` te zijn.

‹ Terug naar de vraag

VRAAG 2

De enum wordt een karakteristieke entiteit `Keuken`, met de attributen `naam`, `land` en `uitleg`, en `Recept` krijgt een foreign key `keukenId`. Een enum kan geen eigen gegevens dragen.

‹ Terug naar de vraag

VRAAG 3

`boolean` en `datetime`. Vegetarisch is ja of nee. Het moment van plaatsen is een datum mét uur.

‹ Terug naar de vraag

10. NA en NNA

BOUWSTEEN

Wat het is

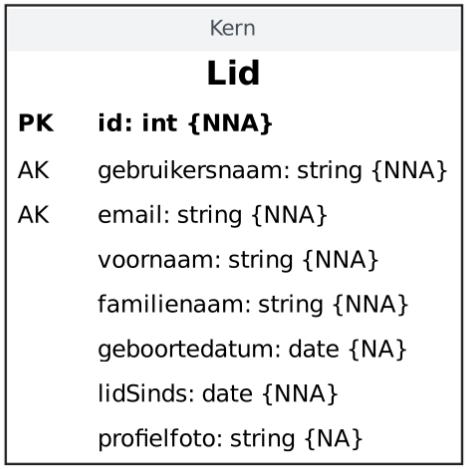
Voor elk attribuut beslis je of het leeg mag blijven.

- **NA**, *Nulls Allowed*: het mag leeg zijn. In de database staat dan `NULL`.
- **NNA**, *Nulls Not Allowed*: het moet altijd ingevuld zijn.

Je noteert het tussen accolades, achter het datatype: `einddatum: date {NA}`.

`NULL` betekent *onbekend* of *niet van toepassing*. Het is iets anders dan `0` of een lege tekst.

Hoe het eruitziet



Lid met NA en NNA bij elk attribuut.

Hoe je beslist

De keuze hangt af van wat je modelleert. Het voorbeeld uit de les is een `Bestuurslid` van een tennisclub.

Attribuut	Keuze	Waarom
`id`	NNA	een primaire sleutel is altijd NNA
`begindatum`	NNA	je kan geen bestuurslid zijn zonder ooit te beginnen
`einddatum`	geen duidelijke keuze	ligt de termijn vast, of blijf je bestuurslid zo lang je wil? Dan is ze NA, want tijdens je termijn is ze nog onbekend
`functie`	NNA	zonder functie betekent het record niets
`spelerId`	NNA	af te lezen uit de relatie

Stel jezelf per attribuut de vraag: **kan dit record bestaan zonder deze waarde?** Kan het, dan is het attribuut NA.

NA en NNA bij een foreign key

Bij een foreign key kies je niet vrij. Je leest het af uit de **aard** van de relatie, aan de kant van de entiteit waarnaar ze verwijst.

Aard aan de 1-kant	Foreign key	Voorbeeld
1, verplicht: `1..1`	NNA	elk recept hoort bij precies één keuken: `keukenId: int {NNA}`
0, optioneel: `0..1`	NA	een recept kan een variant zijn van een ander recept, maar hoeft niet: `origineelId: int {NA}`

Zie hoofdstuk 7 voor de aard.

> **BELANGRIJK**
>
> De primaire sleutel is altijd NNA. Een foreign key volgt altijd de aard van haar relatie. Spreken die twee elkaar tegen, zoals `1..1` en `{NA}`, dan klopt je model niet.

Veelgemaakte fouten

> **LET OP**
>
> - **Alles NNA zetten**, ook wat in werkelijkheid onbekend kan zijn, zoals een einddatum of een tweede adresregel. Dan kan je die records niet bewaren.
> - **Alles NA zetten**, om geen fouten te krijgen. Dan bewaakt je database niets meer.
> - **Een foreign key die de aard tegenspreekt.**
> - **NULL verwarren met 0.** Een onbekend aantal porties is iets anders dan nul porties.

Uitgewerkt voorbeeld: NA of NNA voor elk attribuut

> **SITUATIE**
>
> Een workshop van Smaakvol heeft deze attributen: `id`, `code`, `titel`, `beschrijving`, `datum`, `startuur`, `duurInMinuten`, `maximumDeelnemers` en `prijs`. Daarnaast verwijst ze naar een chef en een thema:
>
> - `R1`: een workshop wordt gegeven door precies één chef. Aan de kant van `Chef` staat `1..1`.
> - `R2`: een workshop kan een thema hebben, maar het hoeft niet. Aan de kant van `Thema` staat `0..1`.
>
> Smaakvol zegt ook: een workshop wordt soms al aangekondigd voor de beschrijving geschreven is. Er bestaan gratis workshops.
>
> Beslis voor elk attribuut: NA of NNA?

Stap 1. De gewone attributen. Stel per attribuut de vraag: *kan een workshop bestaan zonder deze waarde?*

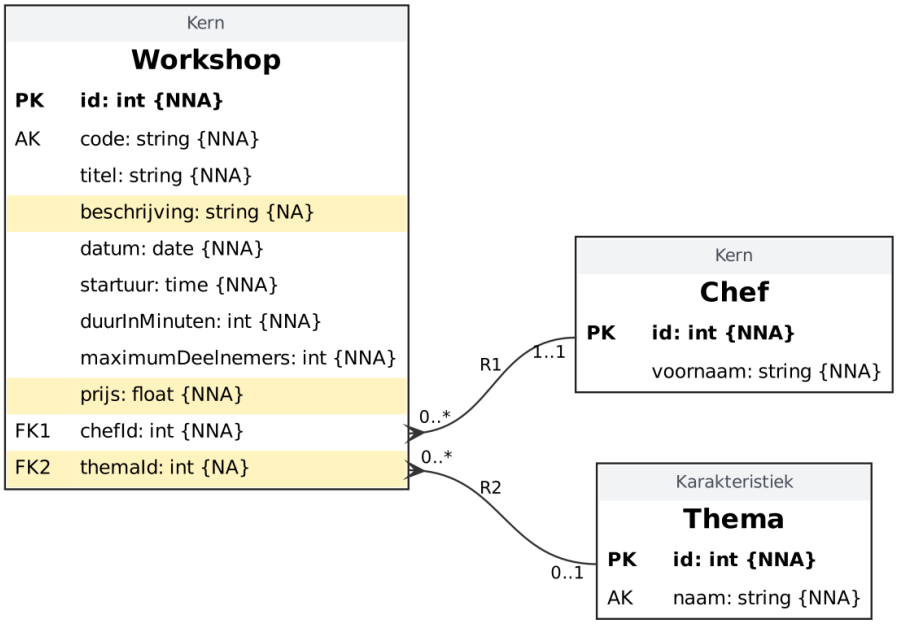
Attribuut	Kan een workshop zonder?	Keuze
`id`	✗ een primaire sleutel is altijd ingevuld	NNA
`code`, `titel`	✗ zonder weet niemand welke workshop het is	NNA
`beschrijving`	✓ een workshop wordt soms aangekondigd voor de beschrijving er is	NA
`datum`, `startuur`, `duurInMinuten`	✗ zonder kan niemand komen	NNA
`maximumDeelnemers`	✗ zonder weet je niet wanneer de workshop vol is	NNA
`prijs`	✗ ook een gratis workshop heeft een prijs: `0`	NNA

Let op bij `prijs`. Een gratis workshop heeft prijs `0`: een gekende waarde. `NULL` zou zeggen dat de prijs onbekend is. Dat is iets anders.

Stap 2. De foreign keys. Hier kies je niet vrij: je leest het af uit de aard aan de kant van de entiteit waarnaar ze verwijst.

Foreign key	Aard aan de 1-kant	Keuze
`chefId`	`1..1` bij `Chef`: verplicht	NNA
`themaId`	`0..1` bij `Thema`: optioneel	NA

Het resultaat.



Workshop met NA en NNA. Gemarkeerd: de drie die NA zijn of opvallen.

Waarom zo.

- *Kan dit record bestaan zonder deze waarde?* Ja: NA. Nee: NNA.
- De primaire sleutel is altijd NNA.
- Een foreign key volgt de aard van haar relatie: aard 1 wordt NNA, aard 0 wordt NA.
- `NULL` is onbekend of niet van toepassing, niet hetzelfde als `0`.

Zo controleer je het zelf.

- Probeer voor elk NNA-attribuut een echte situatie te bedenken waarin de waarde nog niet bestaat. Vind je er een, dan is het NA.
- Leg elke foreign key naast de multipliciteit aan de overkant. Spreken ze elkaar tegen, zoals `1..1` met `{NA}`?
- Staat er ergens NA om fouten te vermijden, terwijl de waarde altijd bestaat?

Voorspel de uitkomst

Vraag 1

`R2` verbindt `Recept` en `Lid`. Aan de kant van `Lid` staat `1..1`. Hoe noteer je de foreign key in `Recept`?

Toon het antwoord ›

Vraag 2

Smaakvol laat leden een profielfoto toevoegen, maar het hoeft niet. Welke keuze maak je voor `profielfoto`?

Toon het antwoord ›

Vraag 3

In `Beoordeling` staat `commentaar: string {NNA}`. Een lid wil een recept enkel een score geven, zonder tekst. Lukt dat?

Toon het antwoord ›

Antwoorden

VRAAG 1

`FK2 lidId: int {NNA}`, met daarbij nog de delete-optie, zoals `{NNA, DTR}`. Aard 1 aan de kant van `Lid` maakt de foreign key NNA. Zie hoofdstuk 11 voor de delete-optie.

‹ Terug naar de vraag

VRAAG 2

NA. Een lid kan bestaan zonder profielfoto.

‹ Terug naar de vraag

VRAAG 3

Nee. NNA eist dat `commentaar` altijd ingevuld is. Wil Smaakvol een score zonder tekst toelaten, dan wordt `commentaar` NA.

‹ Terug naar de vraag

11. DTC, DTR en DTN

BOUWSTEEN

Wat het is

Je verwijdert een keuken, maar er hangen nog recepten aan. Wat moet er met die recepten gebeuren? Dat leg je voor elke foreign key vast, met één van drie **delete-opties**.

Optie	Voluit	Wat er gebeurt als je de rij verwijdert waarnaar verwezen wordt
DTC	*Delete of Target Cascades*	de rijen met een foreign key ernaar worden mee verwijderd
DTR	*Delete of Target Restricted*	het verwijderen lukt niet zolang er een foreign key naar die rij verwijst
DTN	*Delete of Target Nullifies*	de foreign keys die ernaar verwijzen, worden leeggemaakt: `NULL`

Je noteert de optie bij de foreign key, tussen dezelfde accolades als NA of NNA: `keukenId: int {NNA, DTR}`.

Wat er gebeurt met je gegevens

Vertrek van deze twee tabellen. Je verwijdert keuken 1, *Italiaans*.

`Keuken`

id	naam
1	Italiaans
2	Thais

`Recept`

id	titel	keukenId
1	Lasagne	1
2	Pad thai	2
3	Risotto	1
4	Pannenkoeken	NULL

Met DTC verdwijnen ook lasagne en risotto. In `Recept` blijven enkel pad thai en de pannenkoeken over.

Met DTR lukt het verwijderen niet. SQL Server geeft een fout zoals *The DELETE statement conflicted with the REFERENCE constraint "FK_Recept_Keuken"*. Je moet eerst de recepten verwijderen of een andere keuken geven.

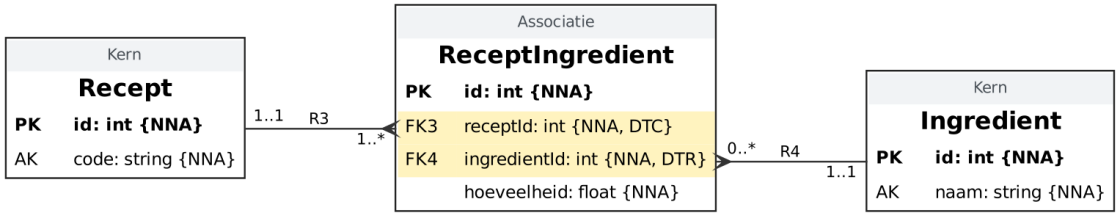
Met DTN blijven alle recepten staan, maar lasagne en risotto krijgen `NULL` als `keukenId`.

Hoe je kiest

Optie	Wanneer	Let op
DTR	de standaard. Je wil niet per ongeluk gegevens verliezen	het standaardgedrag van de meeste DBMS'en
DTC	enkel als het verwijderen van de ouder de kinderen zinloos maakt. Bijna uitsluitend bij een associatie-entiteit	**SQL heeft geen undo.** Wat weg is, is weg
DTN	als de kinderen zonder ouder verder kunnen	kan enkel bij een optionele aard: de foreign key moet NA zijn

Het voorbeeld uit de les voor DTC is een orderlijn: wordt een bestelling verwijderd, dan mogen haar orderlijnen mee. Bij Smaakvol: wordt een recept verwijderd, dan mogen zijn rijen in `ReceptIngredient` mee. Het ingrediënt zelf blijft bestaan.

Redeneer vanuit wat je modelleert. Twijfel je, overleg dan met je opdrachtgever.



ReceptIngredient met DTC naar Recept, en DTR naar Ingredient.

> **BELANGRIJK**
>
> **DTN en NNA gaan niet samen.** DTN maakt de foreign key leeg. Is ze NNA, dan mag dat niet. Een foreign key met `{NNA, DTN}` spreekt zichzelf tegen.

Veelgemaakte fouten

> **LET OP**
>
> - **DTC bij een kernentiteit**, zoals recepten die verdwijnen als je een keuken verwijdert.
> - **DTN bij een verplichte relatie**: `{NNA, DTN}`.
> - **Geen delete-optie** bij een foreign key. Elke foreign key krijgt er een.
> - **Een delete-optie bij een attribuut dat geen foreign key is.**
> - **DTC of DTN bij een recursieve relatie in SQL Server.** Zie hoofdstuk 12.

> **OPMERKING**
>
> In SQL schrijf je de delete-optie bij de foreign key: `ON DELETE CASCADE` voor DTC, `ON DELETE SET NULL` voor DTN, en niets of `ON DELETE NO ACTION` voor DTR. Zie deel 2.
>
> **In EF Core** heet het `DeleteBehavior.Cascade`, `DeleteBehavior.SetNull` en `DeleteBehavior.Restrict`. Zie hoofdstuk 31 van het C#-leerboek.

Uitgewerkt voorbeeld: een delete-optie voor elke foreign key

> **SITUATIE**
>
> Het deel van Smaakvol voor kookworkshops heeft vier foreign keys:
>
> - in `Inschrijving`: `lidId {NNA}` en `workshopId {NNA}`;
> - in `Workshop`: `chefId {NNA}` en `themaId {NA}`.
>
> Wat Smaakvol wil:
>
> - Verlaat een lid Smaakvol, dan verdwijnen zijn inschrijvingen.
> - Een workshop met inschrijvingen mag niet verdwijnen: de leden moeten eerst verwittigd en uitgeschreven worden.
> - Een chef die nog workshops heeft, mag niet verwijderd worden.
> - Verdwijnt een thema, dan blijven de workshops bestaan, zonder thema.
>
> Kies voor elke foreign key DTC, DTR of DTN.

Stap 1. Per foreign key de vraag: wat moet er met de rijen gebeuren die ernaar verwijzen?

Foreign key	Je verwijdert…	Wat moet er gebeuren?	Keuze
`lidId` in `Inschrijving`	een lid	zijn inschrijvingen hebben zonder lid geen betekenis	DTC
`workshopId` in `Inschrijving`	een workshop	niet zolang er inschrijvingen zijn	DTR
`chefId` in `Workshop`	een chef	niet zolang hij workshops heeft. Een workshop is een kernentiteit: die laat je niet mee verdwijnen	DTR
`themaId` in `Workshop`	een thema	de workshops blijven, met `NULL` als thema	DTN

Stap 2. Kijk na of NA en NNA kloppen met de keuze. DTN maakt de foreign key leeg. Dat kan enkel als ze NA is. `themaId` is NA: in orde. Bij de drie andere is DTN uitgesloten, want ze zijn NNA.

Stap 3. Wat er gebeurt met je gegevens. Neem `themaId` met DTN. Je verwijdert thema 2, *Bakken*.

Voor:

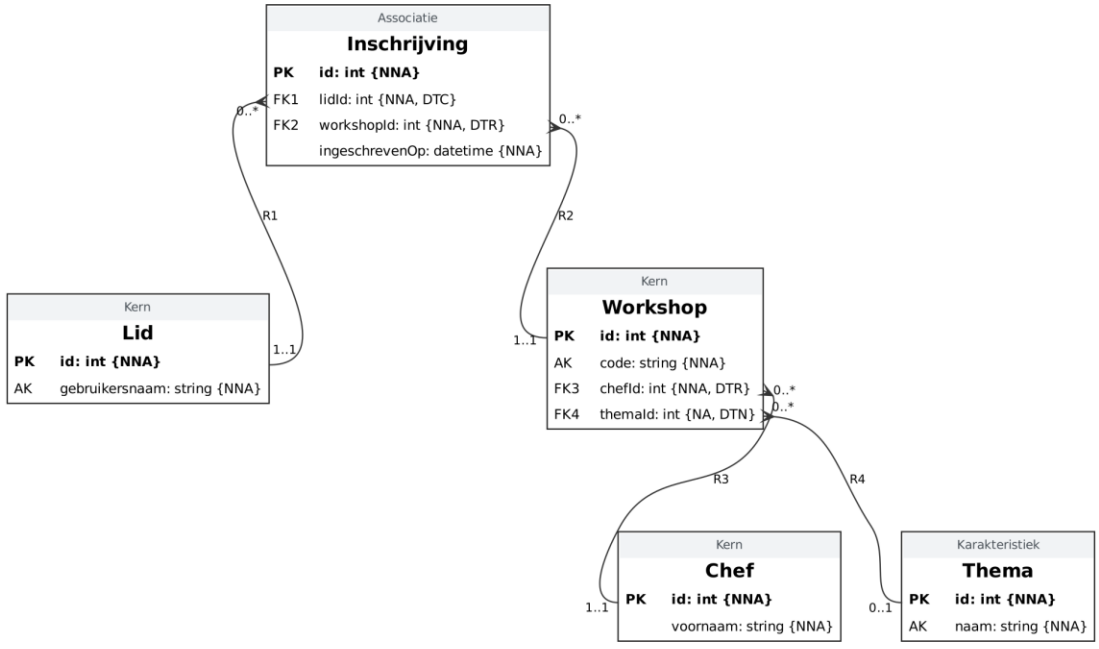
id	code	themaId
1	WS-0141	1
2	WS-0142	2
3	WS-0143	2

Na:

id	code	themaId
1	WS-0141	1
2	WS-0142	NULL
3	WS-0143	NULL

Met DTR zou het verwijderen mislukken, zolang workshop 2 en 3 naar *Bakken* verwijzen. Met DTC zouden workshop 2 en 3 mee verdwijnen. Dat wil Smaakvol niet: een thema weghalen mag geen workshops kosten.

Het resultaat.



Elke foreign key met NA of NNA en haar delete-optie.

Waarom zo.

- DTR is de standaard: je verliest niet per ongeluk gegevens.
- DTC enkel als de kinderen zonder ouder zinloos zijn, bijna uitsluitend bij een associatie-entiteit. SQL heeft geen undo.
- DTN enkel als de kinderen zonder ouder verder kunnen, en de foreign key NA is.

Zo controleer je het zelf.

- Staat er ergens `{NNA, DTN}`? Dat spreekt zichzelf tegen.
- Staat er DTC bij een foreign key in een kernentiteit? Vraag je dan af of die rijen echt mee mogen verdwijnen.
- Heeft elke foreign key een delete-optie, en geen enkel ander attribuut?

Voorspel de uitkomst

Vertrek telkens van de tabellen `Keuken` en `Recept` hierboven.

Vraag 1

`keukenId` is `{NA, DTN}`. Je verwijdert keuken 2, *Thais*. Hoeveel recepten hebben daarna `NULL` als `keukenId`?

Toon het antwoord ›

Vraag 2

`keukenId` is `{NNA, DTR}`. Je probeert keuken 1 te verwijderen. Wat staat er daarna in `Keuken`?

Toon het antwoord ›

Vraag 3

`keukenId` is `{NA, DTC}`. Je verwijdert keuken 1. Hoeveel rijen telt `Recept` daarna?

Toon het antwoord ›

Vraag 4

Een lid kan geen, één of meerdere recepten plaatsen. Elk recept is van precies één lid. Smaakvol wil dat de recepten blijven bestaan als een lid zijn account verwijdert. Welke delete-optie kies je, en wat moet er dan veranderen aan de relatie?

Toon het antwoord ›

Antwoorden

VRAAG 1

Twee. Pad thai krijgt `NULL`, en de pannenkoeken hadden al `NULL`. Lasagne en risotto houden keuken 1.

‹ Terug naar de vraag

VRAAG 2

Beide keukens. Lasagne en risotto verwijzen naar keuken 1, dus DTR blokkeert het verwijderen. Er verandert niets.

‹ Terug naar de vraag

VRAAG 3

Twee rijen: pad thai en de pannenkoeken. Lasagne en risotto zijn mee verwijderd.

‹ Terug naar de vraag

VRAAG 4

DTN, en de aard aan de kant van `Lid` wordt 0: `0..1` in plaats van `1..1`. Dan wordt de foreign key NA, en kan ze leeggemaakt worden. Met `1..1` en NNA kan DTN niet.

‹ Terug naar de vraag

12. Recursieve relaties

SPECIAL TOPIC

Wat het is

Een **recursieve relatie** is een relatie van een entiteit met zichzelf. Je gebruikt ze voor gegevens in een hiërarchie, een boomstructuur:

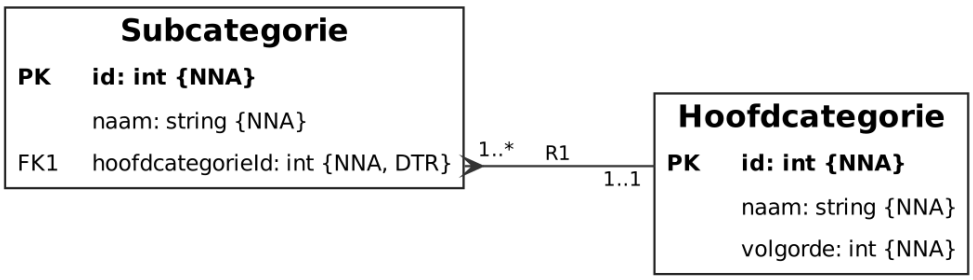
- een organigram: een medewerker en zijn overste;
- categorieën met subcategorieën;
- een stamboom;
- softwarepakketten en de pakketten waarvan ze afhangen.

Bij Smaakvol: de categorie *Hoofdgerecht* heeft subcategorieën *Vlees*, *Vis* en *Vegetarisch*, en *Vlees* heeft zelf weer *Rund* en *Kip*.

De cursus geeft twee manieren om een hiërarchie te modelleren.

Optie 1: een hiërarchisch model

Een entiteit per niveau. De structuur staat volledig in het model.

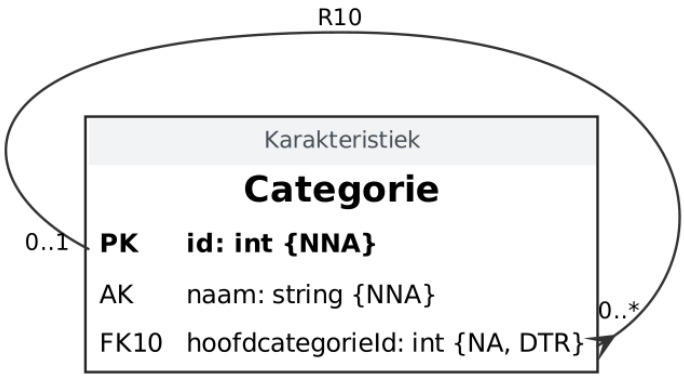


Een entiteit per niveau: Hoofdcategorie en Subcategorie.

Voordelen	Nadelen
een duidelijk model	een groter model
per niveau eigen verplichte relaties en attributen	dezelfde attributen in meerdere entiteiten

Optie 2: een recursieve relatie

Eén algemene entiteit met een relatie naar zichzelf. De foreign key verwijst naar de primaire sleutel van dezelfde entiteit.



Categorie met een recursieve relatie: hoofdcategorieId verwijst naar Categorie zelf.

> ***R10**: Een categorie heeft geen of één hoofdcategorie. Een categorie is de hoofdcategorie van geen, één of meerdere andere categorieën.*

Voordelen	Nadelen
een kleiner model	de flexibiliteit is weg
attributen staan maar in één entiteit	wat NNA was op één niveau, moet nu NA worden. De bovenste categorie heeft geen hoofdcategorie, dus `hoofdcategorieId` moet NA zijn

In het voorbeeld uit de les hadden manager en directeur een `bonus` en een `budget` die NNA waren. In één recursieve entiteit `Werknemer` moeten ze NA worden, want een gewone medewerker heeft ze niet.

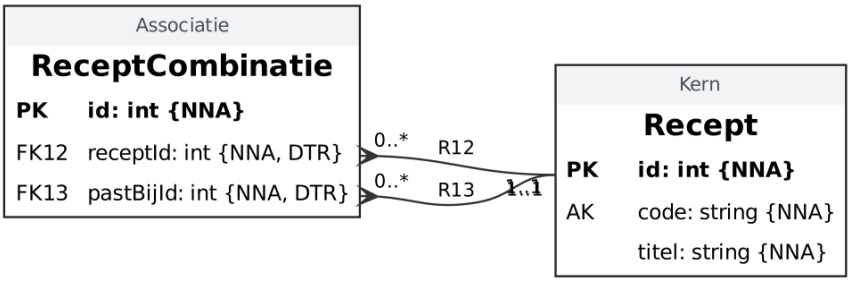
Het model geeft de werkelijkheid ook minder goed weer. Komt er een tweede recursieve relatie bij, zoals een HR-vertegenwoordiger voor elke medewerker, dan kan in het recursieve model ook de directeur er een krijgen, wat in werkelijkheid niet zo is.

Meervoudige recursieve relaties

Soms is het een veel-op-veelrelatie met zichzelf. Het voorbeeld uit de les is een stamboom: een persoon kan geen of meerdere broers en zussen hebben. Of vrienden op een sociaal netwerk.

De oplossing is een **recursieve associatie-entiteit**: een associatie-entiteit met twee foreign keys naar dezelfde entiteit.

Bij Smaakvol: een recept past bij andere recepten, zoals een saus bij een pasta, en een pasta bij meerdere sauzen.



ReceptCombinatie: een associatie-entiteit met twee foreign keys naar Recept.

> **OPMERKING**
>
> Een relationele database is niet echt gemaakt voor netwerken zoals een stamboom of een vriendenkring. Een grafendatabase zoals Neo4j is daar veel handiger voor, zegt de cursus.

Recursieve relaties en verwijderen

Bij een recursieve relatie zou je DTN of DTC willen gebruiken: verwijder je een hoofdcategorie, dan worden de subcategorieën mee verwijderd, of losgekoppeld.

SQL Server laat dat niet toe. Het zou tot lussen of meerdere cascade-paden kunnen leiden. Het voorbeeld uit de les, waarbij `fk` naar `id` in dezelfde tabel verwijst:

id	fk
1	3
2	1
3	2

Rij 1 verwijst naar 3, 3 naar 2, en 2 naar 1. Verwijder je rij 2 met DTC, dan moet 3 mee, en dan 1, en dan weer 2. Dat is een lus.

BELANGRIJK

Gebruik bij een recursieve relatie **DTR** in je model, als je het in SQL Server gaat bouwen.

In PostgreSQL mag `ON DELETE CASCADE` of `SET NULL` wel op een foreign key naar dezelfde tabel. Wat bij SQL Server niet mag, kan daar soms wel.

Veelgemaakte fouten

LET OP

- **De recursieve foreign key NNA zetten.** Dan heeft elke categorie een hoofdcategorie nodig, ook de bovenste. Dat kan niet.
- **DTC of DTN bij een recursieve relatie** in een model voor SQL Server.
- **Een veel-op-veelrelatie met zichzelf zonder associatie-entiteit.**

Uitgewerkt voorbeeld: varianten van een recept

SITUATIE

Leden plaatsen varianten van bestaande recepten. *Vegetarische lasagne* is een variant van *Lasagne*. *Vegan lasagne* is op haar beurt een variant van *Vegetarische lasagne*. Een recept kan meerdere varianten hebben, en is zelf een variant van hoogstens één ander recept. Hoe diep het gaat, ligt niet vast.

Het model wordt gebouwd in SQL Server.

Stap 1. Herken de situatie. Een recept hangt samen met een ander recept: een relatie van `Recept` met zichzelf. Elke variant heeft hoogstens één origineel, en een origineel kan meerdere varianten hebben. Dat is een boom: een hiërarchie.

Stap 2. Optie 1 of optie 2?

	Optie 1: een entiteit per niveau	Optie 2: een recursieve relatie
hoeveel niveaus?	✗ vast: `Origineel`, `Variant`, … Een variant van een variant van een variant past er niet in	✓ zo diep als je wil
zelfde attributen	✗ titel, code en bereidingstijd in elke entiteit opnieuw	✓ één keer, in `Recept`
eigen gegevens per niveau	geen: een variant is gewoon een recept	geen nodig

Het aantal niveaus ligt niet vast, en een variant heeft dezelfde gegevens als elk recept. Je kiest **optie 2**.

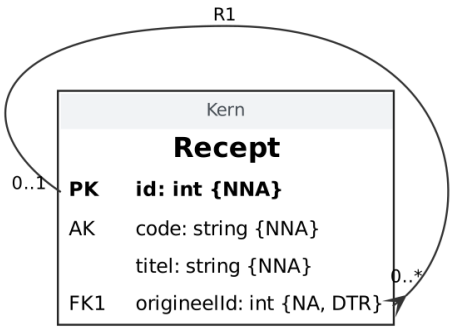
Stap 3. De foreign key. `Recept` krijgt een foreign key naar zichzelf. De naam beschrijft de rol: `origineelId`.

id	titel	origineelId
1	Lasagne	NULL
2	Vegetarische lasagne	1
3	Vegan lasagne	2

Stap 4. NA of NNA? Een origineel heeft geen origineel. `origineelId` moet dus NA zijn. Met NNA zou je *Lasagne* niet kunnen bewaren.

Stap 5. De delete-optie. SQL Server laat DTC en DTN niet toe bij een foreign key naar dezelfde tabel. Je kiest **DTR**: een recept met varianten verwijder je pas als de varianten een ander origineel hebben, of zelf weg zijn.

Het resultaat.



Recept met een recursieve relatie: origineelId verwijst naar Recept zelf.

> ***R1**: Een recept heeft geen, één of meerdere varianten. Een recept is een variant van geen of één ander recept.*

Waarom zo.

- Een recursieve relatie houdt het model klein, en laat zoveel niveaus toe als nodig.
- De recursieve foreign key is NA: bovenaan de boom is er niets om naar te verwijzen.
- In SQL Server: DTR bij een recursieve relatie.

Zo controleer je het zelf.

- Kan je het bovenste recept bewaren? Dan moet de foreign key NA zijn.
- Staat er DTC of DTN bij een recursieve relatie, voor SQL Server? Dat lukt niet.
- Is het een veel-op-veelrelatie met zichzelf, zoals *een recept past bij meerdere andere recepten*? Dan heb je een recursieve associatie-entiteit nodig.

Voorspel de uitkomst

Vraag 1

`hoofdcategorieId` is `{NNA, DTR}`. Je wil de bovenste categorie, *Alle recepten*, toevoegen. Lukt dat?

Toon het antwoord ›

Vraag 2

Smaakvol heeft drie vaste niveaus: *Gang*, *Soort* en *Variant*. Elk niveau heeft eigen verplichte attributen, zoals een volgorde voor de gangen. Welke optie past het best?

Toon het antwoord ›

Vraag 3

Hoeveel foreign keys naar `Recept` heeft `ReceptCombinatie`?

Toon het antwoord ›

Antwoorden

VRAAG 1

Nee. NNA eist een hoofdcategorie, en de bovenste categorie heeft er geen. De foreign key moet NA zijn.

‹ Terug naar de vraag

VRAAG 2

Optie 1, het hiërarchische model. De niveaus liggen vast, en elk heeft eigen verplichte attributen. In één recursieve entiteit zouden die attributen NA moeten worden.

‹ Terug naar de vraag

VRAAG 3

Twee: `receptId` en `pastBijId`. Elk krijgt het nummer van een eigen relatie, en elk verwijst naar `Recept`.

‹ Terug naar de vraag

13. Tijdsafhankelijke attributen

SPECIAL TOPIC

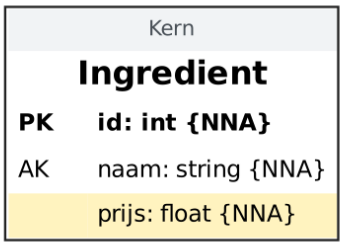
Wat het is

Sommige attributen veranderen in de loop van de tijd, terwijl je de oude waarden wil blijven kennen. Het voorbeeld uit de les is de prijs van een product: een bestelling van vorig jaar moet de prijs van toen tonen, niet die van vandaag.

Bij Smaakvol: de prijs van een ingrediënt, om de kostprijs van een recept te berekenen. Wat kostte dit recept in de winter, en wat nu?

Optie 1: enkel de actuele waarde

Je bewaart enkel de waarde van nu, als gewoon attribuut. Verandert ze, dan overschrijf je ze.



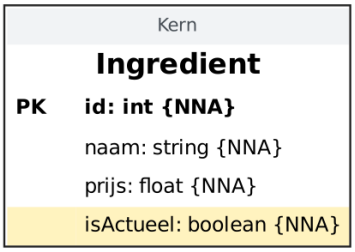
Enkel de actuele prijs, als attribuut van Ingredient.

Eenvoudig, maar de oude waarde is weg. Dat kan als niemand ze nog nodig heeft.

Optie 2: de actuele én de oude waarden

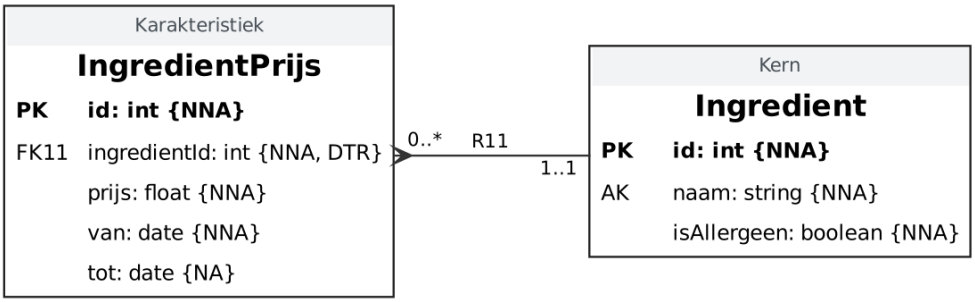
Hoe je dat doet, hangt af van hoe vaak de waarde verandert.

Zeer uitzonderlijk: voeg een booleaans attribuut toe. Een prijswijziging wordt dan een nieuw record. Het oude krijgt de markering dat het niet meer actueel is, en het nieuwe neemt het over. In de les wordt een product met een nieuwe prijs zo een nieuw product.



Een booleaans attribuut: een nieuwe prijs is een nieuw ingrediënt-record.

Frequent: maak een extra **karakteristieke entiteit** met de historische waarden. Elke waarde krijgt een periode: van wanneer tot wanneer ze gold.



IngredientPrijs bewaart elke prijs met haar periode.

> ***R11**: Een ingrediënt heeft geen, één of meerdere prijzen in de tijd. Een prijs hoort bij precies één ingrediënt.*

`tot` is NA: de prijs die nu geldt, heeft nog geen einde. Het voorbeeld uit de les heet `PrijsPerPeriode`, met `van: datetime {NNA}` en `tot: datetime {NA}`.

> **BELANGRIJK**
>
> Kies je optie 2 met een eigen entiteit, dan staat de prijs **niet meer** als gewoon attribuut in `Ingredient`. Anders bewaar je de actuele prijs twee keer: in `Ingredient` en in de rij van `IngredientPrijs` zonder einddatum.

Veelgemaakte fouten

> LET OP
>
> - **De einddatum NNA zetten.** De waarde die nu geldt, heeft nog geen einde.
> - **De actuele waarde twee keer bewaren**, in de entiteit zelf en in de historiek.
> - **Optie 1 kiezen terwijl de oude waarden nodig zijn**, zoals de prijs op een oude factuur.

Uitgewerkt voorbeeld: een prijs die verandert

> SITUATIE
>
> Smaakvol kondigt workshops maanden op voorhand aan, en leden schrijven zich in over die hele periode. Intussen verandert de prijs soms, bijvoorbeeld als de ingrediënten duurder worden. Bij tientallen workshops gebeurt dat geregeld. Smaakvol wil voor elke inschrijving kunnen terugvinden welke prijs toen gold.
>
> Tot nu toe stond `prijs` als gewoon attribuut in `Workshop`.

Stap 1. Is het een tijdsafhankelijk attribuut? De waarde verandert, en de oude waarden zijn nodig. Ja.

Stap 2. Welke optie?

Optie	Past het?	Waarom
1: enkel de actuele waarde	✗	bij een nieuwe prijs is de oude weg, en een oude inschrijving toont de prijs van vandaag
2, met een booleaans attribuut	✗	enkel voor een zeer uitzonderlijke wijziging. Elke prijswijziging zou een nieuw workshop-record maken, met dezelfde code. Dan is `code` geen alternatieve sleutel meer
2, met een eigen entiteit	✓	voor frequente wijzigingen: elke prijs met haar periode

Je kiest de eigen entiteit: `WorkshopPrijs`, een karakteristieke entiteit.

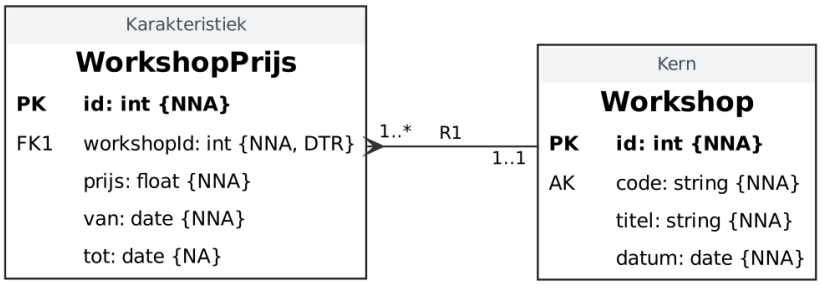
Stap 3. De attributen van `WorkshopPrijs`. De `prijs`, en de periode waarin ze gold: `van` en `tot`. De prijs die nu geldt, heeft nog geen einde: `tot` is NA.

id	workshopId	prijs	van	tot
1	1	24.50	2026-01-01	2026-05-31
2	1	27.00	2026-06-01	NULL

Een inschrijving van 15 maart koppel je aan de prijs waarvan de periode 15 maart bevat: 24.50. Hoe je dat opvraagt, zie je in deel 3.

Stap 4. Haal `prijs` weg uit `Workshop`. Anders staat de actuele prijs twee keer: in `Workshop` en in de rij van `WorkshopPrijs` zonder einddatum.

Het resultaat.



WorkshopPrijs bewaart elke prijs met haar periode.

> ***R1**: Een prijs hoort bij precies één workshop. Een workshop heeft één of meerdere prijzen in de tijd.*

Waarom `1..*` en niet `0..*`? Elke workshop heeft altijd een prijs, ook een gratis workshop: `0`. Er is dus minstens één rij. Een database kan *minstens één* niet afdwingen met een foreign key: dat bewaakt je software. Zie hoofdstuk 7.

Waarom zo.

- Optie 1 als niemand de oude waarde nog nodig heeft.
- Een booleaans attribuut bij een zeer uitzonderlijke wijziging.
- Een eigen entiteit met een periode bij frequente wijzigingen.
- De actuele waarde bewaar je één keer.

Zo controleer je het zelf.

- Is de einddatum NA? De waarde die nu geldt, heeft nog geen einde.
- Staat de waarde nog als attribuut in de oorspronkelijke entiteit? Dan bewaar je ze twee keer.
- Kan je voor een datum in het verleden precies één prijs vinden?

Voorspel de uitkomst

Vraag 1

`IngredientPrijs` heeft voor bloem deze rijen. Welke prijs geldt op 1 maart 2026?

id	ingredientId	prijs	van	tot
1	4	0.89	2025-01-01	2025-12-31
2	4	0.95	2026-01-01	2026-05-31
3	4	1.05	2026-06-01	NULL

Toon het antwoord ›

Vraag 2

En welke prijs geldt vandaag?

Toon het antwoord ›

Vraag 3

De prijs van een kookboek bij Smaakvol verandert één keer in de vijf jaar, en oude bestellingen moeten de oude prijs tonen. Welke variant past het best?

Toon het antwoord ›

Antwoorden

VRAAG 1

0.95. 1 maart 2026 valt tussen 1 januari en 31 mei 2026.

‹ Terug naar de vraag

VRAAG 2

1.05. De rij zonder einddatum is de prijs die nu geldt. Die is van kracht sinds 1 juni 2026.

‹ Terug naar de vraag

VRAAG 3

Optie 2 met een booleaans attribuut. De prijs verandert zelden, dus een nieuwe prijs als nieuw record is eenvoudiger dan een eigen entiteit met perioden. Veranderde ze vaak, dan was een eigen entiteit beter.

‹ Terug naar de vraag

14. Overerving

SPECIAL TOPIC

Wat het is

Soms heb je een algemeen type met meerdere soorten, die gemeenschappelijke gegevens delen en elk ook eigen gegevens hebben. Dat heet **overerving**: het algemene type is het **supertype**, elke soort een **subtype**.

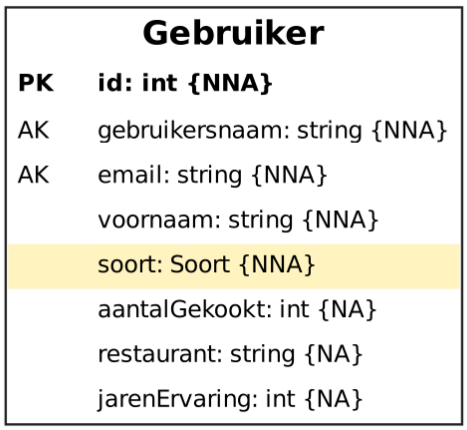
Het voorbeeld uit de les is een webshop met twee soorten werknemers. Van allemaal zijn de NAW-gegevens nodig: naam, adres, woonplaats. Van een magazijnier ook de gewerkte uren en de datum van indiensttreding. Van een technisch medewerker de jaren ervaring, zijn niveau van junior tot lead, en de technologieën die hij kent.

Bij Smaakvol: er zijn twee soorten gebruikers. Van allemaal heb je een gebruikersnaam, een e-mailadres en een voornaam. Een **thuiskok** heeft daarnaast een aantal gekookte recepten. Een **chef** heeft een restaurant en een aantal jaren ervaring.

De cursus geeft drie opties.

Optie 1: één entiteit

Alles in één entiteit. Een **discriminator**, een attribuut dat de soort aangeeft, maakt het onderscheid. De attributen die niet bij elke soort horen, zijn NA.



Gebruiker als één entiteit, met soort als discriminator.

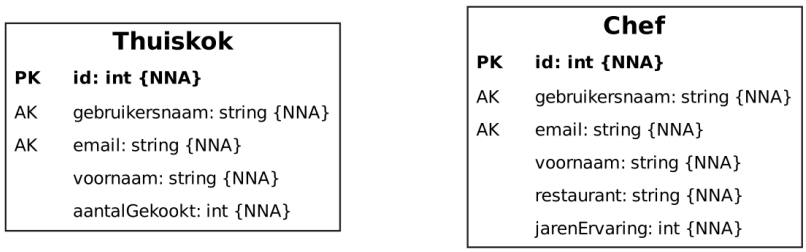
Voordelen	Nadelen
geen joins nodig	de tabel kan groot worden
een wijziging is één `INSERT` of `UPDATE`	veel lege attributen
geen attributen dubbel gedefinieerd	de bedrijfsregels zitten niet in het model: niets verbiedt een thuiskok met een restaurant

Een goede keuze als de soorten weinig eigen attributen hebben, en het onwaarschijnlijk is dat er nog soorten bijkomen of veranderen.

In de les is de discriminator geen tekst, maar een foreign key naar een karakteristieke entiteit `Functie`. Een enum kan ook. Zie hoofdstuk 9.

Optie 2: een entiteit per klasse

Elke soort wordt een eigen entiteit, met alle attributen: de gemeenschappelijke en de eigen.



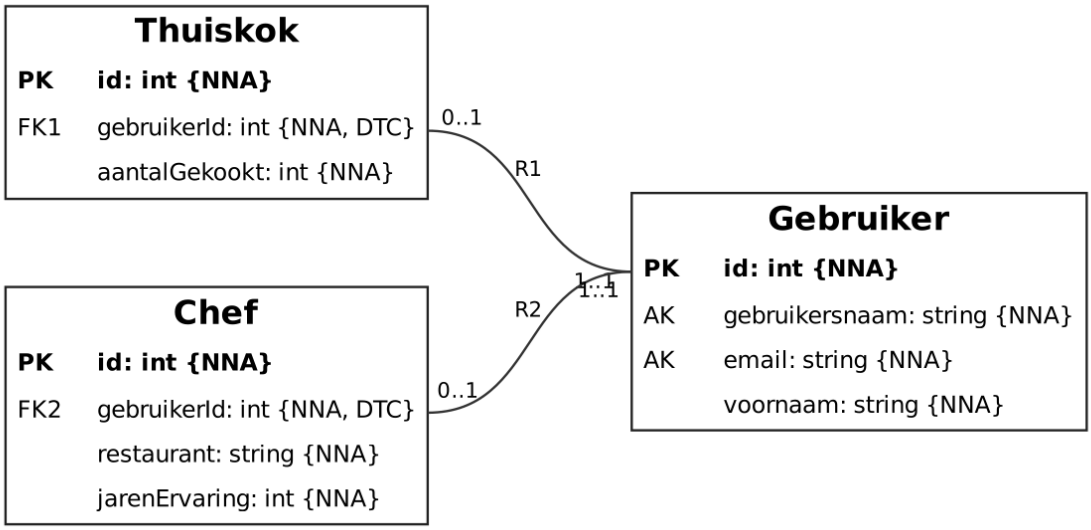
Thuiskok en Chef als aparte entiteiten, elk met alle attributen.

Voordelen	Nadelen
een duidelijker model, zonder NA en zonder discriminator	veel attributen dubbel gedefinieerd
een soort toevoegen zonder de andere aan te passen	veel tabellen
geen joins nodig, een wijziging is één `INSERT` of `UPDATE`	alle gebruikers opvragen vraagt een `UNION`
de bedrijfsregels zitten in het model	

Een goede keuze als er waarschijnlijk nog veel verandert, en de soorten meer eigen dan gemeenschappelijke attributen hebben. De soorten moeten **disjunct** zijn: een gebruiker kan niet tegelijk thuiskok en chef zijn.

Optie 3: joined tables

Een entiteit voor het supertype, met de gemeenschappelijke attributen. Per subtype een entiteit met enkel de eigen attributen, en een foreign key naar het supertype.



Gebruiker als supertype, met Thuiskok en Chef als subtypes.

Voordelen	Nadelen
een soort toevoegen zonder de andere aan te passen	joins nodig om een volledige gebruiker op te vragen
geen attributen dubbel gedefinieerd	een wijziging vraagt meerdere `INSERT`- of `UPDATE`-opdrachten
	bij een grote structuur kan het trager worden

Een goede keuze als de soorten veel eigen attributen hebben, en het erg waarschijnlijk is dat er nog soorten bijkomen of veranderen.

De drie naast elkaar

	Optie 1: één entiteit	Optie 2: per klasse	Optie 3: joined tables
aantal entiteiten	1	1 per soort	1 + 1 per soort
lege attributen	veel	geen	geen
dubbele attribuutdefinities	geen	veel	geen
joins of `UNION` nodig	nee	`UNION` om alles op te vragen	joins
past bij	weinig eigen attributen, weinig verandering	veel verandering, disjuncte soorten	veel eigen attributen, veel verandering

> **OPMERKING**
>
> In C# zou je dit met een abstracte klasse en subklassen schrijven. Zie hoofdstuk 20 van het C#-leerboek. EF Core kent dezelfde drie manieren om dat in tabellen te bewaren: *table per hierarchy* (optie 1), *table per concrete type* (optie 2) en *table per type* (optie 3).

Veelgemaakte fouten

> **LET OP**
>
> - **Optie 1 zonder discriminator.** Dan weet je niet welke soort een record is.
> - **Optie 1 met NNA bij de attributen van één soort.** Dan kan je de andere soort niet bewaren.
> - **Optie 3 met de gemeenschappelijke attributen ook in de subtypes.** Dan definieer je ze dubbel, en verlies je het voordeel van optie 3.
> - **Optie 2 voor soorten die niet disjunct zijn.**

Uitgewerkt voorbeeld: workshops op locatie en online

> **SITUATIE**
>
> Smaakvol geeft workshops op locatie en online. Van elke workshop kent Smaakvol een code, een titel, een datum, een startuur en een duur. Een workshop op locatie heeft daarnaast een zaal en een maximum aantal deelnemers. Een online workshop heeft een link naar de videoles, en geen maximum. Een workshop is altijd het ene of het andere. Andere vormen zijn niet gepland.
>
> Welke optie van overerving kies je?

Stap 1. Herken super- en subtypes. Het supertype is `Workshop`. De subtypes zijn *op locatie* en *online*. Ze zijn **disjunct**: een workshop is nooit allebei.

Stap 2. Tel de attributen.

	Attributen
gemeenschappelijk	`code`, `titel`, `datum`, `startuur`, `duurInMinuten`: vijf
enkel op locatie	`zaal`, `maximumDeelnemers`: twee
enkel online	`link`: één

Veel gemeenschappelijk, weinig eigen.

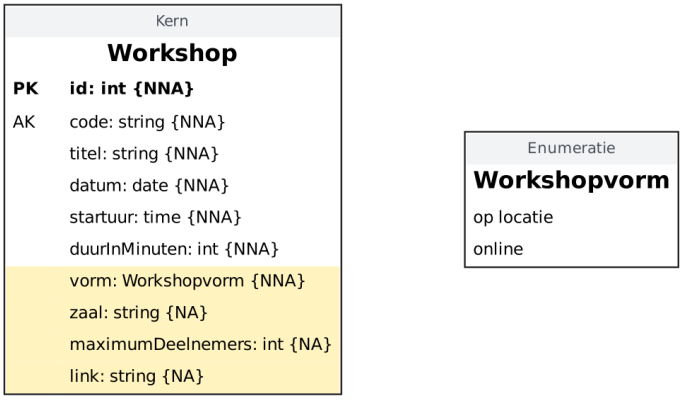
Stap 3. Leg de drie opties naast de situatie.

	Optie 1: één entiteit	Optie 2: per klasse	Optie 3: joined tables
weinig eigen attributen	✓ past	✗ vijf attributen dubbel	½ drie entiteiten voor drie eigen attributen
weinig kans op nieuwe vormen	✓ past	½	½ vooral sterk als er vormen bijkomen
alle workshops samen opvragen, zoals de agenda	✓ geen join	✗ een `UNION`	✓ geen join, zolang de agenda enkel gemeenschappelijke gegevens toont
de regels in het model	✗ niets verbiedt een online workshop met een zaal	✓	½ niets verbiedt een workshop met een rij in beide subtypes

Je kiest **optie 1**: één entiteit, met een **discriminator**. Het nadeel neem je erbij: je software moet bewaken dat een online workshop geen zaal krijgt.

Stap 4. De discriminator en de NA's. Het attribuut `vorm` zegt welke soort een workshop is. Het is een kleine, vaste lijst: een enumeratie `Workshopvorm`, met *op locatie* en *online*. De eigen attributen van één soort worden NA, want de andere soort heeft ze niet.

Het resultaat.



Workshop als één entiteit, met vorm als discriminator. Gemarkeerd: de discriminator en de eigen attributen van elke soort.

Wanneer had je anders gekozen? Komen er geregeld vormen bij, zoals een workshop bij iemand thuis met eigen gegevens, en krijgt elke vorm veel eigen attributen, dan wordt optie 3 de betere keuze.

Waarom zo.

- Optie 1 bij weinig eigen attributen en weinig verandering. Altijd met een discriminator, en de eigen attributen NA.
- Optie 2 bij veel verandering en disjuncte soorten, met meer eigen dan gemeenschappelijke attributen.
- Optie 3 bij veel eigen attributen en veel verandering.

Zo controleer je het zelf.

- Heeft optie 1 een discriminator? Zonder weet je niet welke soort een record is.
- Is een attribuut van één soort NNA? Dan kan je de andere soort niet bewaren.
- Kan je in één zin zeggen waarom je deze optie koos, met de vuistregel erbij?

Voorspel de uitkomst

Vraag 1

In optie 1 zijn `restaurant` en `jarenErvaring` NNA. Je voegt een thuiskok toe. Lukt dat?

Toon het antwoord ›

Vraag 2

Smaakvol wil alle gebruikers opvragen, met hun gebruikersnaam. Bij welke optie heb je een `UNION` nodig?

Toon het antwoord ›

Vraag 3

In optie 3 heeft `Chef` een foreign key `gebruikerId` naar `Gebruiker`. Elke chef is precies één gebruiker, en een gebruiker is hooguit één chef. Welke multipliciteit staat aan de kant van `Chef`, en welke aan de kant van `Gebruiker`?

Toon het antwoord ›

Antwoorden

VRAAG 1

Nee. Een thuiskok heeft geen restaurant. NNA eist dat het ingevuld is. In optie 1 moeten de attributen van één soort NA zijn.

‹ Terug naar de vraag

VRAAG 2

Bij optie 2. Daar staan de gebruikers in twee aparte tabellen, die je met een `UNION` samenvoegt. Bij optie 1 staan ze in één tabel. Bij optie 3 staan de gemeenschappelijke gegevens, zoals de gebruikersnaam, in `Gebruiker`.

‹ Terug naar de vraag

VRAAG 3

`0..1` bij `Chef`, `1..1` bij `Gebruiker`. Een gebruiker is geen of één chef. Een chef is precies één gebruiker. Het is de zeldzame één-op-éénrelatie die hier wel een reden heeft: het subtype hangt aan het supertype.

‹ Terug naar de vraag

15. Je model, les per les

LES PER LES

Wat dit hoofdstuk is

Hoofdstuk 1 tot 14 leggen elke bouwsteen apart uit. Dit hoofdstuk zet ze in de volgorde van de lessen, met de synthesemethode. Het model van Smaakvol groeit mee: na elke les zie je wat erbij komt, en waarom.

Lees de les, en kijk dan hier hoe die leerstof in een volledig model terechtkomt.

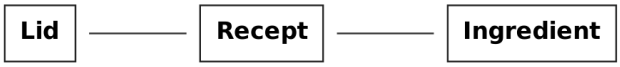
De bron

Dit is de beschrijving waarmee het model begint.

> *Smaakvol is een receptensite. Leden registreren zich met een gebruikersnaam, een e-mailadres, hun voornaam en familienaam. Hun geboortedatum mogen ze invullen. Smaakvol onthoudt sinds wanneer iemand lid is.*
>
> *Een lid plaatst recepten. Een recept heeft een unieke code, een titel, een beschrijving, een bereidingstijd in minuten, een aantal porties, een moeilijkheid (makkelijk, gemiddeld of moeilijk), en het moment waarop het geplaatst werd. Smaakvol toont ook of een recept vegetarisch is. Elk recept hoort bij één keuken, zoals Italiaans of Thais.*
>
> *Een recept bevat ingrediënten, elk in een bepaalde hoeveelheid en eenheid, zoals 250 gram of 2 eetlepels. Van een ingrediënt kent Smaakvol de naam, of het een allergeen is, en het aantal calorieën per 100 gram.*
>
> *Recepten staan in categorieën, zoals Hoofdgerecht of Dessert. Een categorie kan subcategorieën hebben. Leden beoordelen recepten met een score op 5, eventueel met commentaar.*
>
> *Smaakvol volgt de prijs van ingrediënten door de tijd, om te tonen wat een recept kost.*

Na les 1: entiteiten en attributen

De kernentiteiten. Waarover houdt Smaakvol gegevens bij? Over leden, recepten en ingrediënten. Dat is het conceptuele model.

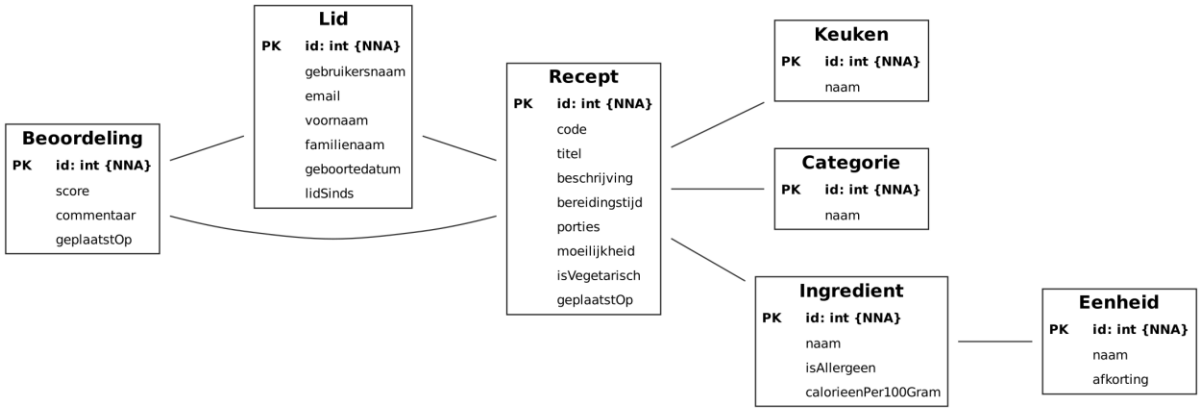


Het conceptuele model: drie kernentiteiten.

De andere entiteiten. Uit de bron komen er nog vier bij: een keuken, een categorie, een beoordeling, en een eenheid. Wat ze precies zijn, beslis je in les 2.

De attributen. Per entiteit wat de bron noemt, met deze beslissingen:

Beslissing	Waarom	Hoofdstuk
`voornaam` en `familienaam`, niet `naam`	een samengesteld gegeven	3
`geboortedatum`, geen leeftijd	een leeftijd is een procesgegeven	3
`bereidingstijd` als één getal in minuten	rekenen en sorteren	3
geen `gemiddeldeScore` in `Recept`	een procesgegeven: je berekent het uit de beoordelingen	3
geen `kostprijs` in `Recept`	ook die bereken je, uit de ingrediënten en hun prijs	3
de naam van de keuken niet in `Recept`	ze hoort bij de keuken, niet bij elk recept opnieuw	3
`titel` in `Recept`, niet `receptTitel`	de naam van de entiteit hoort niet in het attribuut	4



Na les 1: de entiteiten met hun attributen, en lijnen voor wat samenhangt.

De lijnen zeggen enkel wat samenhangt. Multipliciteiten, sleutels en beschrijvingen komen in les 2.

Na les 2: sleutels, relaties en soorten

Sleutels. Elke entiteit krijgt `id` als primaire sleutel. De kernentiteiten krijgen een alternatieve sleutel: `code` voor een recept, `gebruikersnaam` en `email` voor een lid, `naam` voor een ingrediënt. Ook keukens, eenheden en categorieën hebben een unieke naam.

De veel-op-veelrelaties. Er zijn er drie, en elk wordt een associatie-entiteit:

Veel-op-veel	Wordt	Eigen attributen
recepten en ingrediënten	`ReceptIngredient`	`hoeveelheid`, en de eenheid
recepten en categorieën	`ReceptCategorie`	geen
leden en recepten die ze beoordelen	`Beoordeling`	`score`, `commentaar`, `geplaatstOp`

De eenheid hoort bij de combinatie van recept en ingrediënt: bloem staat in het ene recept in gram, in het andere in kopjes.

De soorten. `Lid`, `Recept` en `Ingredient` zijn kernentiteiten. `ReceptIngredient`, `ReceptCategorie` en `Beoordeling` zijn associatie-entiteiten. `Keuken`, `Eenheid` en `Categorie` zijn karakteristiek: ze bestaan om een recept of een ingrediëntregel te beschrijven.

De relaties. Genummerd, met de foreign key aan de veel-kant.

Relatie	Tussen	Multipliciteit	Foreign key
R1	`Recept` en `Keuken`	`0..*` — `1..1`	`FK1 keukenId` in `Recept`
R2	`Recept` en `Lid`	`0..*` — `1..1`	`FK2 lidId` in `Recept`
R3	`ReceptIngredient` en `Recept`	`1..*` — `1..1`	`FK3 receptId`
R4	`ReceptIngredient` en `Ingredient`	`0..*` — `1..1`	`FK4 ingredientId`
R5	`ReceptIngredient` en `Eenheid`	`0..*` — `1..1`	`FK5 eenheidId`
R6	`Beoordeling` en `Lid`	`0..*` — `1..1`	`FK6 lidId`
R7	`Beoordeling` en `Recept`	`0..*` — `1..1`	`FK7 receptId`
R8	`ReceptCategorie` en `Recept`	`0..*` — `1..1`	`FK8 receptId`
R9	`ReceptCategorie` en `Categorie`	`0..*` — `1..1`	`FK9 categorieId`

De eerste multipliciteit staat aan de kant van de eerste entiteit. `R3` heeft `1..*`: elk recept heeft minstens één ingrediëntregel.

De relatiebeschrijvingen.

- **R1**: Een keuken heeft geen, één of meerdere recepten. Een recept hoort bij precies één keuken.
- **R2**: Een lid plaatst geen, één of meerdere recepten. Een recept is geplaatst door precies één lid.
- **R3 en R4**: Een recept bevat één of meerdere ingrediënten, en een ingrediënt komt in geen, één of meerdere recepten voor.
- **R5**: Een eenheid wordt gebruikt in geen, één of meerdere ingrediëntregels. Een ingrediëntregel heeft precies één eenheid.
- **R6 en R7**: Een lid beoordeelt geen, één of meerdere recepten, en een recept wordt beoordeeld door geen, één of meerdere leden.
- **R8 en R9**: Een recept staat in geen, één of meerdere categorieën, en een categorie bevat geen, één of meerdere recepten.

R3 en R4 beschrijven samen één veel-op-veelrelatie. Die mag je in één keer beschrijven. Zie hoofdstuk 7.

Na les 3: datatypes, NA en NNA, delete-opties

Datatypes. Logische, algemene types. Getallen enkel waar Smaakvol rekent.

Attribuut	Type	Waarom
`code`, `gebruikersnaam`, `email`	`string`	tekst, ook als er cijfers in staan
`bereidingstijd`, `porties`	`int`	om te rekenen en te sorteren
`hoeveelheid`	`float`	0,5 liter, 2,5 eetlepel
`isVegetarisch`, `isAllergeen`	`boolean`	ja of nee
`geplaatstOp`	`datetime`	een datum met uur
`geboortedatum`, `lidSinds`	`date`	een datum zonder uur
`score`	`int`	1 tot 5

Zelfgemaakte types. De moeilijkheid is een enumeratie: drie waarden, en daar komt niets bij. Keuken en eenheid zijn karakteristieke entiteiten: er komen er bij, en een eenheid heeft ook een afkorting. Zie hoofdstuk 9.

NA. Enkel wat de bron als optioneel beschrijft: `geboortedatum` en `commentaar`. Alle foreign keys hebben aard 1 aan de 1-kant, dus allemaal NNA.

Delete-opties.

Foreign key	Optie	Waarom
`keukenId` in `Recept`	DTR	een keuken met recepten mag niet verdwijnen
`lidId` in `Recept`	DTR	een lid met recepten kan zijn account niet zomaar verwijderen
`receptId` in `ReceptIngredient`	DTC	zonder recept zijn de regels zinloos
`ingredientId` in `ReceptIngredient`	DTR	een ingrediënt in gebruik mag niet verdwijnen
`eenheidId` in `ReceptIngredient`	DTR	idem voor een eenheid
`lidId` en `receptId` in `Beoordeling`	DTC	zonder lid of recept is een beoordeling zinloos
`receptId` in `ReceptCategorie`	DTC	de koppeling verdwijnt met het recept
`categorieId` in `ReceptCategorie`	DTR	een categorie met recepten mag niet verdwijnen

DTC staat enkel bij associatie-entiteiten. DTN komt niet voor: geen enkele foreign key is NA.

Na les 12: special topics

Recursieve relatie. Categorieën hebben subcategorieën, en die kunnen weer subcategorieën hebben. Het aantal niveaus ligt niet vast, en elk niveau heeft dezelfde gegevens. Daarom optie 2: `Categorie` krijgt `hoofdcategorieId`, een foreign key naar zichzelf.

- **R10**: Een categorie heeft geen of één hoofdcategorie. Een categorie is de hoofdcategorie van geen, één of meerdere categorieën.
- `hoofdcategorieId: int {NA, DTR}`: NA, want de bovenste categorieën hebben geen hoofdcategorie. DTR, want SQL Server laat bij een recursieve relatie geen DTC of DTN toe. Zie hoofdstuk 12.

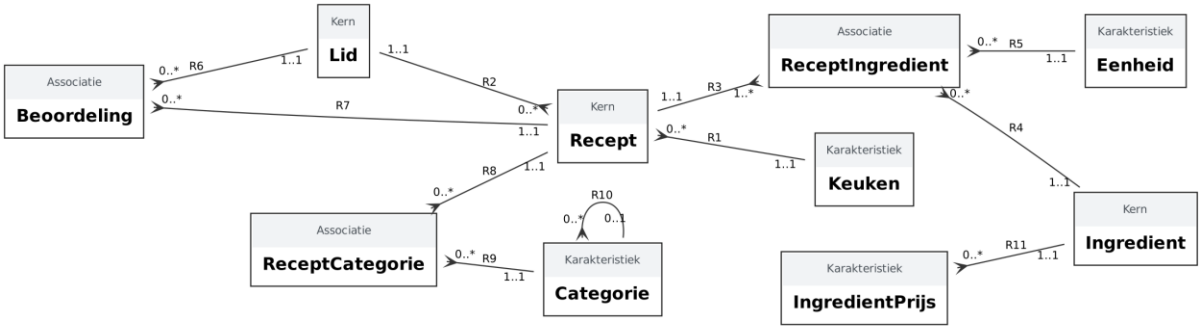
Tijdsafhankelijk attribuut. Prijzen van ingrediënten veranderen vaak. Daarom optie 2 met een eigen entiteit: `IngredientPrijs`, met een prijs en een periode. De prijs staat niet meer in `Ingredient`. Zie hoofdstuk 13.

- **R11**: Een ingrediënt heeft geen, één of meerdere prijzen door de tijd. Een prijs hoort bij precies één ingrediënt.

Overerving. Komt in Smaakvol niet voor: er is één soort lid. Kwam er een onderscheid tussen thuiskoks en chefs, dan was hoofdstuk 14 aan de beurt.

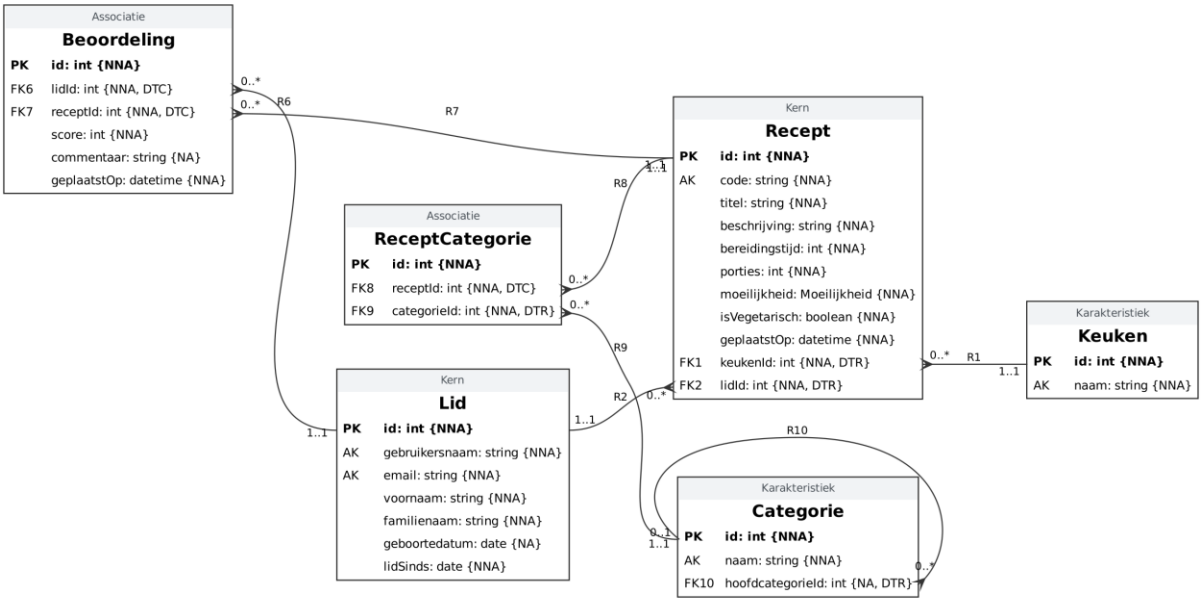
Het volledige model

Eerst het overzicht: elke entiteit met haar soort, en elke relatie met haar nummer en multipliciteit.

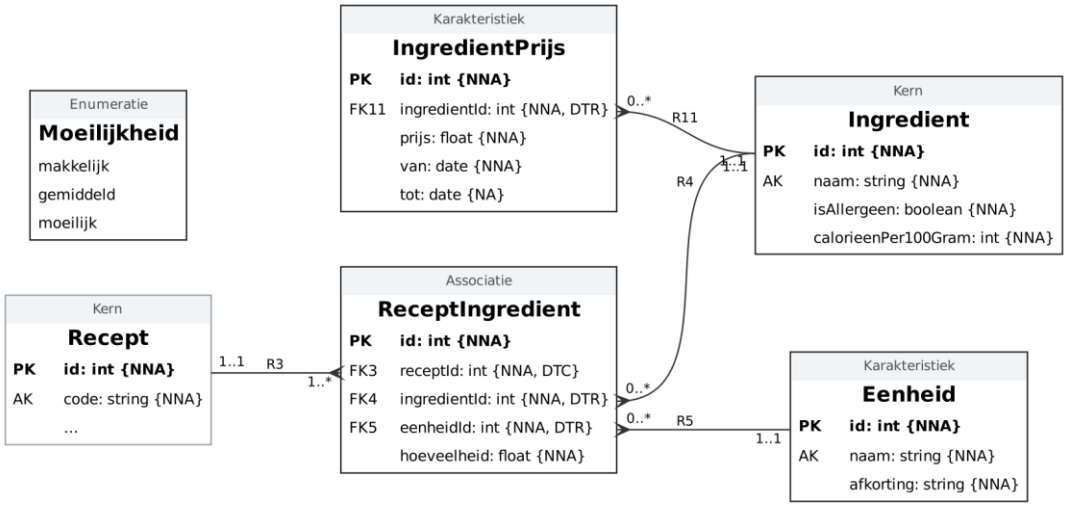


Smaakvol na les 12: het overzicht.

Dan het model in twee delen, met alle attributen. `Recept` staat in beide: rond het recept en zijn leden, en rond zijn ingrediënten.



Deel A: recepten, leden, beoordelingen, categorieën en keukens.



Deel B: ingrediënten, eenheden en prijzen, en de enumeratie Moeilijkheid.

> **OPMERKING**
>
> De enumeratie `Moeilijkheid` staat apart in het model, als lijst met haar drie waarden. Ze heeft geen relatie: het is geen entiteit.

Alles nagekeken

> **CHECKLIST**
>
> Uit de checklists van les 1 en 2, en de feedbackcriteria van de cursus:
>
> - Elke entiteit begint met een hoofdletter en staat in het enkelvoud. Attributen beginnen met een kleine letter, in camelCase.
> - Elke entiteit heeft `id` als primaire sleutel. Elke kernentiteit heeft minstens één alternatieve sleutel.
> - Elke relatie is genummerd. Elke foreign key heeft het nummer van haar relatie.
> - Elke foreign key staat aan de `*`-kant, en bevat de volledige primaire sleutel van de 1-kant.
> - Geen veel-op-veelrelatie zonder associatie-entiteit.
> - Elke relatie heeft een multipliciteit aan beide kanten, en een beschrijving die ermee klopt.
> - Elke entiteit heeft haar soort.
> - Elk attribuut heeft een datatype en NA of NNA. Elke foreign key volgt de aard van haar relatie.
> - Elke foreign key heeft een delete-optie. Geen DTN bij NNA. Geen DTC of DTN bij een recursieve relatie in SQL Server.
> - Geen samengestelde gegevens, geen procesgegevens.

Voorspel de uitkomst

Vertrek telkens van het volledige model.

Vraag 1

Lid 5 heeft twee recepten geplaatst en drie recepten beoordeeld. Je probeert lid 5 te verwijderen. Wat gebeurt er?

Toon het antwoord ›

Vraag 2

Je verwijdert recept 12. Het heeft vier ingrediëntregels, staat in twee categorieën, en heeft drie beoordelingen. Wat verdwijnt er mee, en wat blijft?

Toon het antwoord ›

Vraag 3

Welke foreign keys in het volledige model zijn NA?

Toon het antwoord ›

Vraag 4

Smaakvol wil bewaren welke recepten een lid als favoriet aanduidt, en sinds wanneer. Wat komt er bij in het model?

Toon het antwoord ›

Antwoorden

VRAAG 1

Het verwijderen lukt niet, en er verandert niets. `lidId` in `Recept` is DTR, en lid 5 heeft nog recepten. Ook de beoordelingen blijven staan: het verwijderen gaat als geheel niet door, dus de DTC van `Beoordeling` komt niet aan de beurt.

‹ Terug naar de vraag

VRAAG 2

De vier ingrediëntregels, de twee koppelingen met een categorie en de drie beoordelingen verdwijnen mee: hun `receptId` is telkens DTC. De ingrediënten, de categorieën en de leden zelf blijven bestaan.

‹ Terug naar de vraag

VRAAG 3

Eén: `hoofdcategorieId` in `Categorie`. Alle andere relaties hebben aard 1 aan de 1-kant.

‹ Terug naar de vraag

VRAAG 4

Een associatie-entiteit `Favoriet`, met `lidId`, `receptId` en `toegevoegdOp`, en twee nieuwe relaties, R12 en R13. Een lid heeft meerdere favorieten, en een recept is de favoriet van meerdere leden: dat is veel-op-veel. Zie hoofdstuk 8.

‹ Terug naar de vraag

Veelgemaakte verwarringen

Begrippen die op elkaar lijken, telkens kort naast elkaar. Het hoofdstuk erbij legt het volledig uit.

Data tegenover informatie

Data zijn losse gegevens, zonder uitleg. Informatie krijg je als je data ordent en interpreteert. Een database bewaart data. Hoofdstuk 1.

Conceptueel tegenover logisch

Conceptueel toont enkel de kernentiteiten en wat samenhangt. Logisch voegt attributen toe, en eventueel sleutels en algemene datatypes. Hoofdstuk 2.

Logisch tegenover fysiek

Logisch is onafhankelijk van een DBMS, en mag nog veel-op-veelrelaties bevatten. Fysiek is klaar voor één DBMS: alle sleutels, NA en NNA, delete-opties, en geen veel-op-veel meer. Hoofdstuk 2.

Entiteit tegenover attribuut

Een entiteit is iets waarover je gegevens bijhoudt, en wordt een tabel. Een attribuut is één kenmerk ervan, en wordt een kolom. Kan iets meerdere waarden hebben per record, dan is het geen attribuut. Hoofdstuk 3.

Samengesteld gegeven tegenover procesgegeven

Een samengesteld gegeven bestaat uit delen die je apart wil gebruiken: splits het. Een procesgegeven kan je berekenen: bewaar het niet. Hoofdstuk 3.

Primaire sleutel tegenover foreign key

De primaire sleutel identificeert een rij in haar eigen tabel, en heet `id`. Een foreign key verwijst naar de primaire sleutel van een andere tabel, en heet zoals die tabel met `Id` erachter. Hoofdstuk 5 en 6.

Kandidaatsleutel tegenover alternatieve sleutel

Een kandidaatsleutel is alles wat de primaire sleutel zou kunnen zijn. Een alternatieve sleutel is een kandidaatsleutel die je niet als primaire sleutel koos. Hoofdstuk 5.

Natuurlijke sleutel tegenover surrogaatsleutel

Een natuurlijke sleutel komt uit de werkelijkheid, zoals een productcode. Een surrogaatsleutel heeft geen betekenis, zoals `id`. In de cursus is de primaire sleutel altijd een surrogaatsleutel. Hoofdstuk 5.

R1 tegenover FK1

`R1` is het nummer van een relatie, bij de lijn. `FK1` is de foreign key van die relatie, in de entiteit aan de veel-kant. Ze hebben hetzelfde nummer. Hoofdstuk 6.

Aard tegenover connectiviteit

De aard is het eerste getal: 0 is optioneel, 1 is verplicht. De connectiviteit is het tweede: 1 of veel. Hoofdstuk 7.

`0..*` tegenover `1..*`

Allebei veel. `0..*` laat ook nul toe, `1..*` eist er minstens één. Hoofdstuk 7.

Kernentiteit tegenover karakteristieke entiteit

Een kernentiteit bestaat op zichzelf, en is waar je toepassing over gaat. Een karakteristieke entiteit bestaat enkel om een andere te beschrijven, vaak als keuzelijst. Hoofdstuk 8.

Karakteristieke entiteit tegenover associatie-entiteit

Een karakteristieke entiteit beschrijft één andere entiteit. Een associatie-entiteit koppelt er twee, in een veel-op-veelrelatie. Hoofdstuk 8.

Enumeratie tegenover karakteristieke entiteit

Een enumeratie is een vaste, kleine lijst zonder eigen entiteit. Een karakteristieke entiteit is een eigen entiteit, waar waarden en gegevens bij kunnen komen. Hoofdstuk 9.

Logisch datatype tegenover fysiek datatype

Logisch is algemeen: `string`, `int`, `date`. Fysiek is van één DBMS: `varchar(50)`, `int`, `datetime2`. Hoofdstuk 9.

NA tegenover DTN

NA zegt dat een foreign key leeg mag zijn. DTN maakt ze leeg als de rij waarnaar ze verwijst, verdwijnt. DTN kan enkel bij NA. Hoofdstuk 10 en 11.

NULL tegenover 0

`NULL` is onbekend of niet van toepassing. 0 is een gekende waarde. Hoofdstuk 10.

DTC tegenover DTR

DTC verwijdert de rijen die verwijzen mee. DTR blokkeert het verwijderen zolang er rijen verwijzen. Hoofdstuk 11.

Hiërarchisch model tegenover recursieve relatie

Hiërarchisch: een entiteit per niveau. Recursief: één entiteit die naar zichzelf verwijst. Hoofdstuk 12.

Booleaans attribuut tegenover historiek-entiteit

Bij een zeldzame wijziging volstaat een booleaans attribuut en een nieuw record. Bij frequente wijzigingen bewaar je elke waarde met haar periode in een eigen entiteit. Hoofdstuk 13.

Eén entiteit tegenover joined tables

Eén entiteit: alles samen, met een discriminator en veel NA. Joined tables: een supertype met de gemeenschappelijke attributen, en per soort een entiteit met de eigen. Hoofdstuk 14.

Index

Alfabetisch. Het **vette** nummer is het hoofdstuk met de volledige uitleg. De andere nummers zijn hoofdstukken waar je het ook tegenkomt.

Naam	Hoofdstuk
`0..*, 1..1, …`	**7**
aard	**7**, 10, 15
`AK`	**5**, 15
alternatieve sleutel	**5**, 15
associatie-entiteit	6, **8**, 15
attribuut	2, **3**, 4
bondigheid	**1**
`boolean`	**9**, 13
camelCase	3, **4**
conceptueel model	**2**, 15
connectiviteit	**7**
data	**1**
datamodel	**2**
datatype	2, **9**, 15
`date`	**9**
`datetime`	**9**
DBMS	**1**, 2
delete-optie	**11**, 15
discriminator	**14**
disjunct	**14**
`DTC`	**11**, 12, 15
`DTN`	10, **11**, 12
`DTR`	**11**, 12, 15
één-op-één	**6**, 14
één-op-veel	**6**
entiteit	1, **3**, 8
enumeratie	**9**, 15
ERD	1, **2**
ERD-checklist	**4**, 5, 6, 15
`FK`	**6**, 10, 11

Naam	Hoofdstuk
`float`	**9**
foreign key	1, **6**, 10, 11
fysiek model	**2**, 9
grafendatabase	**12**
hefboomeffect	**1**
hiërarchisch model	**12**
index	**5**
informatie	**1**
`int`	5, **9**
joined tables	**14**
kandidaatsleutel	**5**
karakteristieke entiteit	**8**, 9, 13
kernentiteit	2, **8**, 15
kolom	**1**
logisch model	**2**, 9
meervoudige recursieve relatie	**12**
multipliciteit	**7**
`NA`	2, **10**, 15
naamgeving	**4**
natuurlijke sleutel	**5**
`NNA`	2, **10**, 15
`{NNA, DTR}`	**11**
`NULL`	1, **10**, 11
overerving	**14**
`PK`	1, **5**
PostgreSQL	**4**, 9, 12
primaire sleutel	1, **5**
procesgegeven	**3**, 15
`R1, FK1`	**6**
recursieve associatie-entiteit	**12**
recursieve relatie	**12**, 15
relatie	2, **6**
relatiebeschrijving	**7**, 15
relationele database	**1**

Naam	Hoofdstuk
rij	**1**
samengesteld gegeven	**3**, 15
`string`	**9**
subtype	**14**
supertype	**14**
surrogaatsleutel	**5**
synthesemethode	**2**, 15
tabel	**1**
tijdsafhankelijk attribuut	**13**, 15
`UNION`	**14**
`UNIQUE`	**5**
veel-op-veel	2, **6**, 8, 15